ARGUS Master Reference

How it works, how to run it, every important script, and the tests. Built 2026-10-05 from docs/ on the feat/threat-detection branch.

Contents

PART I · TECHNICAL REFERENCE

1 · System overview
- What it does, in one pass
- The four ARGUS classes
- Processes and threads on a running node
- The machines
- Three Flower servers, and which to use
- Repository layout

2 · How to run everything
- 2.1 One-time setup
- 2.2 The node
- 2.3 Checking what the detector sees
- 2.4 Labelling and training on the node
- 2.5 A federated round on one node
- 2.6 The central aggregator and scheduled rounds
- 2.7 A live multi-client simulation (server + Flower clients)
- 2.8 Making detection faster
- 2.9 Exposing the node beyond the LAN
- 2.10 The product dashboard
- 2.11 Tests
- 2.12 Building the documentation

3 · Computer vision: camera to boxes to clips
- 3.1 The frame path
- 3.2 src/camera/platform_detect.py: which board is this?
- 3.3 src/camera/backends.py: one class per board family
- 3.4 src/camera/__init__.py: the camera service
- 3.5 src/detection/worker.py: the background loop
- 3.6 src/detection/__init__.py: the detection cascade
- 3.7 src/detection/annotate.py: drawing the boxes
- 3.8 src/detection/event_recorder.py: clips around events
- 3.9 Other detection files
- 3.10 Measured performance

4 · Classification: the threat model
- 4.1 Where the model comes from
- 4.2 src/detection/threat.py
- 4.3 How the operating point was chosen

4.4 Cost, and when to turn it on
4.5 Tools for this stage
4.6 The tests

5 · Training and federated learning

5.1 The data path, end to end
5.2 features.py: what the head sees
5.3 collector.py: one event per track, not per frame
5.4 store.py: the sample store
5.5 head.py: the model
5.6 model_state.py: the validation gate
5.7 Local training: POST /api/federated/head/train
5.8 A federated round: rounds.py and node_client.py
5.9 scheduler.py: the cron
5.10 robust.py and robust_strategy.py: Byzantine-robust aggregation
5.11 The aggregator: central_server/fl_aggregator.py
5.12 The simulator: sim/
5.13 Results so far
5.14 Datasets: the CIFAR-10 stand-in and what replaced it

6 · API, data and security

6.1 Startup order (src/api/app.py, startup_event)
6.2 Routes
6.3 The database (src/database/__init__.py)
6.4 Security (src/security/__init__.py, src/api/auth.py)
6.5 The built-in dashboard (static/index.html)

7 · Scripts, tools and legacy code

7.1 Runners (scripts/)
7.2 Data and labelling
7.3 Models: fetch, export, accelerate
7.4 Measurement
7.5 Documentation
7.6 The simulator (sim/)
7.7 Legacy code: present, tested in places, not on the running path
7.8 Research prototypes (research/)
7.9 Developer tooling inside the app

8 · Configuration reference

8.1 Server
8.2 Camera
8.3 Detection
8.4 Threat stage
8.5 Recording
8.6 Federated learning (node)
8.7 Security
8.8 Deprecated
8.9 The aggregator (central_server/fl_aggregator.py)
8.10 The legacy server (server/config.py)

9 · Tests

9.1 Running them
9.2 How the suite stays fast and hermetic
9.3 What each file covers
9.4 What is not tested

10 · Status, known gaps, and what to do next

10.1 What works and has been run
10.2 Gaps that block the research claim
10.3 Gaps in the running system
10.4 Housekeeping
10.5 The plan of record (from the 2026-10-04 status report)

PART II · THE GUIDES

1 · What ARGUS is

The problem
What it does, in order
The pieces
What you see
What makes it different
Honest limits, as of now

2 · Setting up the Pi

Before you start
Step 1 - system packages · PI
Step 2 - get the code · PI
Step 3 - Python environment · PI
Step 4 - check the camera · PI
Step 5 - make detection fast · PI
Step 6 - set a password · PI
Step 7 - start it · PI
Step 8 - confirm it is alive · PI
Step 9 - look at it
If the Pi and your laptop are on different networks

3 · Running the demo

Rehearse this once before the day. The first run always finds something.
Shell 1 - start the camera · PI
Shell 2 - the live numbers · PI
The screen - open the page
The walkthrough
What to say if asked
Before you present

4 · The dashboards

The built-in page - nothing to install
The product dashboard - runs on your laptop
Things that commonly go wrong
Which to use when

5 · What it detects, and how to test it
The four things it knows
How something becomes an event
Testing what it sees
Making props work
What gets saved
6 · How the learning works
The idea in one paragraph
What exactly is being learned
Where the answers come from
A round, step by step
The two halves
Running one by hand · PI
Training on saved video · PI
Watch the frame rate while it trains
About the practice data
What is not solved yet
7 · When things break
Start here · PI
The camera
Detection
Speed
The dashboard
Learning
Scripts will not run
Last resort
8 · For developers
Two repositories
Layout
Running it
Settings that change behaviour
Design decisions worth knowing before changing things
What has actually been measured
Open work
Documentation
PART III · QUICK START
ARGUS quick start
1 · PI - start the node
2 · Watch it
3 · PI - check what the camera sees
4 · PI - a federated round
4b · The central server, and the schedule
5 · LAPTOP - the product dashboard (optional)
If it breaks
When the card goes bad

ARGUS node simulator

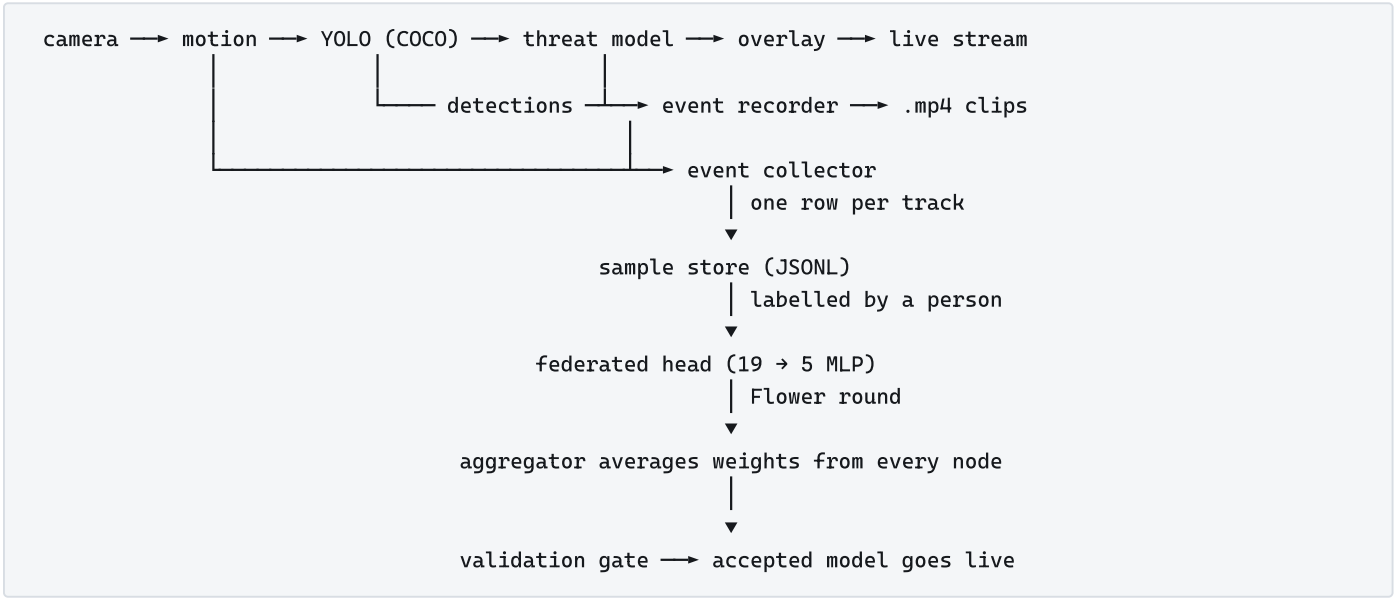
The question it answers
Why it is not just a spreadsheet
Profiles
The federated head
Reading the output
What this does not model
Files

1 · System overview

ARGUS is a camera node that detects what is in front of it, decides whether that is routine for the place it is watching, records the moments that matter, and learns from other nodes without any of them sharing footage.

This reference describes the code on `feat/threat-detection` as of 2026-10-05 (commit `a5f2c39` plus the working-tree changes listed in section 10.4). That branch is the real system. `main` and `master` on GitHub are older and do not have most of what follows.

What it does, in one pass



Three kinds of model run on a node, and only one of them learns:

Model	Question it answers	Where it comes from	Learns on the node?
YOLOv8n (COCO)	What is in the frame: person, vehicle, animal, package?	Ultralytics pretrained weights	No, frozen
Threat model (YOLO11s, 2 classes)	Is this person carrying a weapon?	Third-party thesis weights, AGPL-3.0	No, frozen
Federated head (numpy MLP, ~200K params)	Is this detection routine *for this site*?	Trained on this node's labelled events, averaged across nodes	**Yes**

The federated head is the project's research contribution. YOLO already says "person"; the head learns that a person at the side gate at 03:00 is not routine here, even if it is routine somewhere else.

The four ARGUS classes

YOLO emits 80 COCO classes. ARGUS keeps the ones that matter at a front door and folds them into four (`src/detection/__init__.py`, `_YOLO_CLASS_MAP`):

ARGUS class	COCO classes behind it
person	person
vehicle	bicycle, car, motorcycle, bus, train, truck
animal	bird, cat, dog, horse, sheep, cow
package	backpack, handbag, suitcase

COCO has no parcel class. A cardboard box detects as nothing; a backpack detects as a package. Real parcel detection needs a custom-trained model.

The head then labels each event with one of five outcomes: `routine_person`, `routine_vehicle`, `routine_animal`, `routine_package`, `anomaly`.

Processes and threads on a running node

`python main.py` (or `scripts/run_node.sh`, which wraps it) starts one FastAPI process. Inside it:

Thread / task	Code	Job
uvicorn event loop	`src/api/app.py`	Serves the API, the built-in dashboard, the MJPEG stream, the WebSocket
`argus-capture`	`src/detection/worker.py`	Reads the camera into a one-frame slot; feeds every frame to the recorder
`argus-detect`	`src/detection/worker.py`	Runs the detection cascade on the newest frame only
FL scheduler task	`src/federated/scheduler.py`	Wakes hourly; runs a round every 14 days at 02:00 (only if `FL_ENABLED`)

A federated round never trains inside that process. It launches `python -m src.federated.node_client` as a separate OS process so training competes with detection through the OS scheduler, pinned to one BLAS thread, and a crash in training cannot take the camera down.

The machines

Machine	Runs	Ports
Raspberry Pi 5 + AI HAT+ 2 (the node)	`main.py` via `scripts/run_node.sh`	8000 (API + dashboard)
Laptop (the aggregator)	`central_server/fl_aggregator.py` via `scripts/run_central_server.ps1`	8080 (Flower gRPC), 8090 (REST + `/metrics`)
Laptop (optional)	The Next.js product dashboard, `adam9798/argus-dashboard`	3000 / 3001
Any machine	The node against a video file (`scripts/mock_demo.py`) or the simulator (`sim/`)	8000, 8080, 9108

Only weights cross the network to the aggregator. No video, no images.

Three Flower servers, and which to use

The repository has three ways to start a Flower server. They are not interchangeable.

Server	File	Strategy	Use it for
Central aggregator	`central_server/fl_aggregator.py`	`ArgusStrategy` (robust aggregation + checkpoints + metrics), Flask REST API on 8090	Real rounds between real nodes. This is the one a deployed node talks to.
Simulator server	`sim/fl.py serve`	Robust aggregation (`--aggregation fedavg` for the baseline)	Quick local rounds, `scripts/fl_trial.sh`, and multi-client simulations
Legacy server	`server/app.py`	FedAvg or FedProx	The original YOLO-weights design. Not used by the current node.

The first two, and the minimal `src/federated/server.py`, share one strategy class: `RobustFedAvg` in `src/federated/robust_strategy.py` (section 5.10). It screens every update through `src/federated/robust.py` before averaging. Plain FedAvg stays available behind one switch (`aggregation="fedavg"`), so the comparison study runs both arms through identical server code.

Repository layout

```
main.py                node entry point (uvicorn on src.api.app)
src/
  config.py            every setting, read from environment variables
  camera/              board detection and capture backends
  detection/           motion, YOLO, threat, overlay, worker, recorder
  federated/           features, collector, store, head, gate, rounds,
                       node client, scheduler, robust aggregation
  api/                 routes, auth, dashboard contract, trial endpoints
  database/            SQLite: faces, events, devices, automations, audit, users
  security/            PBKDF2 hashing, bearer tokens, audit log
  automation/          device and rule models (stubbed)
  agents/              error-routing helper agents (dev tooling)
  training/            LEGACY: YOLO fine-tuning dataset and trainer
central_server/        the aggregator, Prometheus, Grafana, tunnel config
server/                LEGACY Flower server
sim/                   node simulator and synthetic multi-site workload
scripts/               runners, exports, benchmarks, tools
deploy/                systemd unit and settings template for a node
static/index.html      the built-in live dashboard
tests/                 328 tests, pytest
docs/                  this reference, guides, runbooks, proposal
research/              WiFi CSI, sensor fusion, early FL prototype
prometheus/            voice assistant orchestrator (separate tool)
```

Legacy code is described in section 7 so it is not mistaken for the running path.

2 · How to run everything

Every block is labelled with the machine it runs on. **PI** is a shell on the Raspberry Pi. **LAPTOP** is Windows (Git Bash or PowerShell, as marked). **ANY** runs on either.

Unless a step says otherwise, you are in the repository root on branch `feat/threat-detection`.

2.1 One-time setup

LAPTOP (Git Bash)

```
git clone https://github.com/Gvictome/argus.git
cd argus
git checkout feat/threat-detection
python -m venv .venv
source .venv/Scripts/activate
pip install -r requirements.txt
```

`requirements.txt` includes `ultralytics`, `flwr` and `pytest-asyncio`. Leaving any of them out shows up later in confusing ways: without `ultralytics` the detector silently finds nothing, and without `pytest-asyncio` four scheduler tests fail.

Fetch the threat-model weights (not committed to git; checksum-verified):

```
python scripts/fetch_models.py
```

Get the sample video the mock demo and one test use:

```
mkdir -p media
curl -fsSL -o media/vtest.avi https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/vtest.avi
```

PI

Full walk-through with explanations: guide 2, *Setting up the Pi*.

```
sudo apt update
sudo apt install -y git python3-venv python3-dev python3-picamera2 python3-opencv build-essential cmake
git clone https://github.com/Gvictome/argus.git ~/argus
cd ~/argus && git checkout feat/threat-detection
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python -c "from picamera2 import Picamera2; print('camera library ok')"
python scripts/export_cpu_models.py --format ncnn
python scripts/fetch_models.py
python scripts/set_password.py --user admin
```

`--system-site-packages` is required: `picamera2` comes from `apt`, and a plain venv cannot see it. If the camera import complains about numpy, `pip install --ignore-installed "numpy>=2.2,<3"`.

2.2 The node

PI

```
./scripts/run_node.sh
```

Flag	Effect
`--every-frame`	Run YOLO on every frame, not only on motion (classifies subjects that hold still)
`--threat`	Turn on the dangerous-person stage (about 100 ms per frame on the Pi CPU)
`--model yolov8s`	A bigger, slower detector; needs its own export to be accelerated
`--video media/vtest.avi`	Use a recording instead of the camera
`--no-record`	Do not write event clips
`--port 8001`	Serve on a different port

`run_node.sh` sets demo-friendly defaults: `AUTH_REQUIRED=false`, `FL_ENABLED=false`, `THREAT_ENABLED=false`, `RECORD_EVENTS=true`, `MOTION_GATE=true`. Note that `THREAT_ENABLED` defaults to **true** in `src/config.py`; `run_node.sh` is what turns it off. Running `python main.py` directly gives you the config defaults instead.

Watch it from a second shell:

```
./scripts/node_status.sh          # one line per second
./scripts/node_status.sh --once
```

Open `http://<pi-address>:8000/dashboard` in a browser. Find the address with `hostname -I`.

ANY: the node against a video, with isolated data

```
python scripts/mock_demo.py --reset            # media/vtest.avi, data in data/mock
python scripts/mock_demo.py --source 0         # a webcam
python scripts/mock_demo.py --threat --record  # threat stage and clips on
```

Everything is real except the camera (a file) and the data directory (`data/mock`). The admin password is `argus-demo` unless `--password` is given. Use this to work on the pipeline without a Pi.

2.3 Checking what the detector sees

ANY

```
python scripts/check_classes.py --camera --save /tmp/seen.jpg   # through the running node's API
python scripts/check_classes.py media/vtest.avi                  # a video
python scripts/check_classes.py photo.jpg                        # an image
```

Prints the ARGUS class and the raw COCO label behind it. This separates "the model missed it" from "the model saw it and it maps somewhere else". `--camera` goes through the node's API because the detection worker owns the camera and a second reader gets no frames.

2.4 Labelling and training on the node

Events land unlabelled. Labels come from a person, through the dashboard (confirm or correct an event) or through the API:

```
curl -s localhost:8000/api/federated/samples | python -m json.tool | head -40
curl -sX POST localhost:8000/api/federated/samples/<sample_id>/label \
     -H 'Content-Type: application/json' -d '{"label": 0}'
```

`label` is an integer: 0 `routine_person`, 1 `routine_vehicle`, 2 `routine_animal`, 3 `routine_package`, 4 `anomaly`. The dashboard route `POST /api/events/{n}/label` takes `{"action": "confirm"}` or `{"action": "correct", "label": "<free text>"}` instead, and reads intent from the words (see section 6).

Turn a saved clip into labelled events in one go (**stop the node first**, both write the same store):

```
python scripts/ingest_video.py media/events/some_clip.mp4 --label routine_person
python scripts/ingest_video.py doorbell.mp4 --label anomaly --every-frame
```

Train the head locally and gate it in (needs 200 labelled samples by default):

```
curl -sX POST localhost:8000/api/federated/head/train \
     -H 'Content-Type: application/json' -d '{"epochs": 6}' | python -m json.tool
```

Body fields: `epochs` (default 8), `min_samples` (200), `tolerance` (0.01).

Not enough labels yet? Seed a synthetic corpus. These rows are tagged `synthetic` and never appear in the event list; say so when presenting.

```
curl -sX POST localhost:8000/api/federated/samples/bootstrap \
     -H 'Content-Type: application/json' -d '{"site": "driveway", "n": 4000}'
```

2.5 A federated round on one node

PI (node already running in another shell)

```
./scripts/fl_trial.sh                  # 3 rounds, seeded corpus
./scripts/fl_trial.sh --rounds 5
./scripts/fl_trial.sh --no-bootstrap   # only real labelled events
```

It starts a local aggregator (`sim/fl.py serve`), seeds samples, trains, joins the round, and prints the gate's decision. `model_version` moves to `fl-r1` , `fl-r2` , ... only when the gate accepts the averaged model.

With one node, FedAvg averages that node with itself. This proves the plumbing, not federation. Federation needs two or more clients: see 2.7.

2.6 The central aggregator and scheduled rounds

LAPTOP (PowerShell)

```
.\scripts\run_central_server.ps1
.\scripts\run_central_server.ps1 -Rounds 3 -MinClients 2
```

It prints the two lines to paste on the Pi, keeps every global model in `central_server/checkpoints` , and stays up between sessions (`ARGUS_SERVE_FOREVER=true`). If the Pi cannot connect, Windows Firewall is the usual cause; the script prints the one-line rule to allow it.

Watch	URL
Server state	`http://localhost:8090/api/status`
Rounds, loss, accuracy	`http://localhost:8090/api/training/summary`
Registered nodes	`http://localhost:8090/api/nodes`
Prometheus metrics	`http://localhost:8090/metrics`

The full stack with Prometheus and Grafana runs under Docker:

```
cd central_server && docker compose up -d     # Grafana on :3000 (admin/argus)
```

Pi: join on a schedule

```
export FL_ENABLED=true
export FL_SERVER_URL=<laptop-ip>:8080
export FL_CENTRAL_API=http://<laptop-ip>:8090
./scripts/run_node.sh
curl -s localhost:8000/api/federated/status | python -m json.tool
```

Rounds then run every 14 days at 02:00, and catch up promptly if the Pi was off across the due date. To run the node as a service that starts on boot, use `deploy/argus-node.service`; the full production setup (schedule, model versions, rollback, monitoring) is in `docs/PRODUCTION_FL.md`. The scheduler has an "only while idle" check, but nothing feeds it motion yet (`notify_motion()` has no caller), so today it always reads idle; see section 10. Training runs in a separate one-core process either way, so detection keeps going. Trigger a round by hand at any time:

```
curl -sX POST localhost:8000/api/federated/round -H 'Content-Type: application/json' \
     -d '{"server": "<laptop-ip>:8080", "epochs": 4}'
curl -s localhost:8000/api/federated/round | python -m json.tool
```

2.7 A live multi-client simulation (server + Flower clients)

Real Flower gRPC server and clients as separate processes on one machine. Each client is a synthetic site with its own non-IID data (`sim/workload.py`).

ANY: four terminals

```
python sim/fl.py serve --rounds 5 --clients 3          # terminal 1, waits for 3 clients
python sim/fl.py client --site driveway                # terminal 2
python sim/fl.py client --site backdoor                # terminal 3
python sim/fl.py client --site yard                    # terminal 4
```

Sites: `driveway`, `backdoor`, `yard`. Client flags: `--samples` (default 6000), `--epochs` (4), `--seed`, `--address`. The server prints per-round distributed loss, accuracy and balanced accuracy at the end.

Verified on 2026-10-02 (3 clients, 3 rounds, 3000 samples, 2 epochs):

Round	Loss	Accuracy	Balanced accuracy
1	0.327	0.918	0.820
2	0.286	0.919	0.829
3	0.279	0.923	0.838

These are synthetic sites whose generator and model were both written by the team. They show the federation works end to end; they are not evidence about real-world accuracy.

ANY: one node under load while a round runs

```
python sim/run_node.py                                # one node at 12 FPS, one round
python sim/run_node.py --clients 3 --rounds 6         # real rounds across 3 clients, robust aggregation
python sim/run_node.py --profile pi5-threat-on-cpu    # the risky configuration
python sim/run_node.py --pipelined --cores 4          # overlap the accelerator, Pi core count
```

It runs the detection loop at a target frame rate, fires a real Flower round against it, and reports what training did to the frame rate. Metrics are on `http://127.0.0.1:9108/metrics` (`--port 0` disables). Profiles and their sources: section 5 and `sim/README.md`.

2.8 Making detection faster

Where	Command	Result
PI	`python scripts/export_cpu_models.py --format ncnn`	NCNN export in `models/`, picked up automatically
x86	`python scripts/export_cpu_models.py --format openvino`	OpenVINO export, the fast x86 CPU path
x86_64 Linux / WSL2 only	`python scripts/export_hailo.py --hw-arch hailo10h`	Hailo export directory for the AI HAT+ 2; copy the **whole directory** to the Pi's `models/`
Jetson Orin only	`python scripts/export_tensorrt.py`	TensorRT engine, built on the device itself

The backend is chosen at startup in this order: TensorRT engine, Hailo export, CPU export (NCNN on ARM, OpenVINO on x86), plain PyTorch. Check which one loaded with `curl -s localhost:8000/api/detection/status`.

An export has a fixed input size. The node forces `OBJECT_IMGSZ` to the export's own size, because asking a 640 export for 416 returns no boxes at all, quickly, which looks like a huge speedup.

2.9 Exposing the node beyond the LAN

```
AUTH_REQUIRED=true ARGUS_ADMIN_PASSWORD='<a real password>' SECRET_KEY='<random>' python main.py
./scripts/run_tunnel.sh --check     # preflight only
./scripts/run_tunnel.sh             # preflight, then cloudflared
```

The tunnel script refuses to start unless the running node reports `AUTH_REQUIRED` on **and** an unauthenticated request to a protected endpoint returns 401. Add a Cloudflare Access policy as the first gate. Details: `docs/CLOUDFLARE_TUNNEL.md`.

2.10 The product dashboard

LAPTOP (PowerShell)

```
.\scripts\run_dashboard.ps1 -Node 10.0.0.140
```

Clones `adam9798/argus-dashboard` if missing, installs it, points it at the node, and starts it on `http://localhost:3000` (or 3001). If it says SYSTEM OFFLINE, the browser cannot reach the Pi, or the port Next.js chose is missing from `CORS_ORIGINS`. The node's own page, `http://<pi>:8000/dashboard`, has no such dependency.

2.11 Tests

ANY

```
python -m pytest -q
python -m pytest tests/test_threat.py -q          # one file
python -m pytest -q -k "gate"                     # by name
```

328 tests, about 60 to 90 seconds on the development laptop. Gate any push on the exit code, not on reading the output. Details: section 9.

2.12 Building the documentation

ANY (needs Google Chrome installed)

```
python scripts/build_docs_pdf.py --master                    # this reference + guides, one PDF
python scripts/build_docs_pdf.py --guides                    # the 8 guides, one PDF each
python scripts/build_docs_pdf.py                             # quick start, overview, runbook...
python scripts/build_docs_pdf.py --master --out-dir "C:/Users/you/Dropbox/argus"
```

Markdown in `docs/` is the source of truth. PDFs are rebuilt from it.

3 · Computer vision: camera to boxes to clips

This section covers every module between the image sensor and the saved footage. The threat model is in section 4; turning detections into training data is in section 5.

3.1 The frame path

```
CaptureBackend.read()          src/camera/backends.py     one BGR frame
        │
CameraService.get_frame_array  src/camera/__init__.py     rotation, colour swap
        │
DetectionWorker._capture_loop  src/detection/worker.py    latest-frame slot ──► EventRecorder.process
        │                                                                        (every frame, for pre-
roll)
DetectionWorker._detect_loop   src/detection/worker.py    newest frame only
        │
DetectionService.process_frame src/detection/__init__.py
        ├─ detect_motion       always
        ├─ detect_objects      on motion, or every frame if MOTION_GATE=false
        ├─ detect_threats      same gate as objects, if a ThreatClassifier is attached
        └─ collector.observe   every frame (section 5)
        │
stream_from_worker             src/detection/worker.py    draw_detections → resize → JPEG → MJPEG
```

Every frame inside ARGUS is a numpy array in **BGR** order, OpenCV's convention. Boxes are `(x, y, width, height)` in pixels unless a function says otherwise.

3.2 `src/camera/platform_detect.py`: which board is this?

`detect_board()` reads `/proc/device-tree/model` and `/etc/nv_tegra_release` and returns `Board.RASPBERRY_PI`, `Board.JETSON` or `Board.GENERIC`.

Why it exists: the old check, `platform.machine().startswith('aarch')`, is also true on a Jetson Orin. The node would then import `picamera2` on a Jetson, fail, and report a camera fault that was really a software mistake. CPU architecture is not the board.

Module-level constants `BOARD`, `IS_RASPBERRY_PI`, `IS_JETSON` are re-exported from `src.camera`.

3.3 `src/camera/backends.py`: one class per board family

Every backend implements the same three methods, `open() -> bool`, `read() -> ndarray | None`, `close()`, and none of them raise.

Backend	Board	How it reads
`Picamera2Backend`	Raspberry Pi	`picamera2` / libcamera; must come from `apt`, needs a venv with `--system-site-packages`
`JetsonCSIBackend`	Jetson Orin	GStreamer `nvarguscamerasrc` pipeline built by `build_csi_pipeline()`; the only way to reach a CSI sensor through the Orin's ISP
`OpenCVBackend`	anything else, and any file	`cv2.VideoCapture`; a video file loops at the end, and with `realtime=True` is paced to its own frame rate

`build_backend(board, sensor_id, config)` picks one. **An explicit `CAMERA_SOURCE` wins over the board**: a file path or a numeric index always gets `OpenCVBackend`, which is how the whole pipeline runs against a recording on a laptop.

`rotation_to_flip_method()` maps 0/90/180/270 degrees onto the GStreamer flip codes.

3.4 `src/camera/__init__.py`: the camera service

Name	Role
`CameraConfig`	resolution, framerate, rotation, `sensor_id`, name, `swap_rb`, `source`, `realtime`
`CameraService`	Wraps one backend. `initialize()`, `get_frame_array()` (BGR array), `get_frame()` (JPEG bytes), `capture_snapshot()`, `start_recording()` / `stop_recording()`, `get_status()`
`CameraRegistry`	Several `CameraService`s by name, for the Orin's two CSI connectors. `build_registry_from_settings()` parses `CAMERA_SENSORS="0:front,1:back"`
`camera_service`	Module singleton for camera 0, used by the detection worker and the single-camera routes

`CAMERA_SWAP_RB=true` swaps red and blue. Use it only if people look blue on the live feed; the Pi camera already returns BGR through the backend.

3.5 `src/detection/worker.py`: the background loop

`DetectionWorker(camera, detector, recorder)` owns the camera. Nothing else may read it: a second reader gets empty frames. Anything that needs a picture asks the worker via `latest()`.

Two daemon threads:

- `argus-capture` opens the camera on its own thread (so startup never blocks on hardware), then loops: read a frame, store it in a one-frame slot under a lock, pass it to `EventRecorder.process()` together with the most recent detections. The recorder needs every frame, not only the detected ones, or its pre-roll would have gaps.
- `argus-detect` takes the newest frame, runs `process_frame()`, stores the detections. Frames that arrived while it was busy are **dropped**, not queued, so the boxes always describe what is in front of the camera now rather than seconds ago.

`status()` reports `capture_fps`, `detect_fps` (30-sample sliding window), `detect_ms` (exponential average), `frames_captured`, `frames_detected`, `frames_not_detected`, and `camera_error` if the camera never opened.

`stream_from_worker(worker, jpeg_quality=80, max_fps=15, stream_width=960)` is the MJPEG generator behind `/api/camera/stream`. It never runs the detector: it draws the worker's latest boxes on the worker's latest frame, downscales to 960 px wide (a 1080p JPEG per frame was the most expensive CPU stage left), encodes, and yields multipart parts. A viewer therefore costs only draw plus encode.

`src/detection/stream.py` holds the older generator, `stream_annotated_mjpeg()`, which detected inside the stream loop. It is kept for its tests and for `BOUNDARY`; the running node uses the worker.

3.6 `src/detection/__init__.py`: the detection cascade

Types

- `DetectionType`: `MOTION`, `HUMAN`, `FACE`, `ANIMAL`, `VEHICLE`, `PACKAGE`, `UNKNOWN`, `DANGEROUS_PERSON` (produced only by the threat stage).
- `Detection`: `type`, `confidence`, `bbox (x, y, w, h)`, `label`, `face_id`, `coco_id`. The raw COCO id is kept because "package" is recognised by id downstream.
- `DetectionConfig`: thresholds, `object_imgsz`, `motion_width`, `motion_gate`, `cpu_export`, `model_name`, `faces_enabled`. The app builds it from `Settings` at startup; before that fix, the dataclass defaults silently beat every environment variable.

`DetectionService.initialize()`: choosing a backend

Probes files, fastest first, and loads the first one present:

1. `models/<model>.engine` (TensorRT, Jetson) → backend `tensorrt`
2. A Hailo export (`models/yolov8n_hailo_model`, `models/yolo11n_hailo_model`, or a bare `.hef`) → backend `hailo`
3. A CPU export (`models/<model>_ncnn_model` first on ARM, `models/<model>_openvino_model` first on x86) → backend `cpu-ncnn` / `cpu-openvino`
4. `<model>.pt` with PyTorch → backend `cpu`

For Hailo and CPU exports it reads the export's `metadata.yaml` and overrides `object_imgsz` with the export's fixed input size, logging a warning. A failed load is logged at ERROR with the traceback and leaves `backend="none"`; it is loud on purpose because a silent fallback disables detection without saying so.

If `ultralytics` is not installed, no model loads and object detection returns nothing. This is the first thing to check when "nothing is ever detected".

`detect_motion(frame)`

Frame differencing on a downscaled copy (`MOTION_WIDTH`, default 320 px): grayscale, Gaussian blur (kernel scaled with the image), `absdiff` against the previous blurred frame, threshold at `MOTION_SENSITIVITY` (25), dilate, external contours. Each contour bigger than 30×30 px (in full-resolution coordinates) becomes a `MOTION` detection whose confidence is contour area over box area. The first call returns nothing because there is no previous frame.

Downscaling and reusing the previous blurred frame took 1080p motion from 17.8 to 22.9 FPS on the development laptop.

`detect_objects(frame)`

One YOLO call with `imgsz=object_imgsz`, `conf=DETECTION_THRESHOLD` (0.5) and `classes=` the 16 COCO ids ARGUS maps, so other classes are dropped before NMS rather than after. Each box is converted from centre format to `(x, y, w, h)` and typed through `_YOLO_CLASS_MAP` (table in section 1).

`detect_threats(frame)`

Delegates to the attached `ThreatClassifier` (section 4). Any exception is caught and logged; a failure in the newest stage must not take down object detection.

`process_frame(frame)`

```
motion = detect_motion(frame)
if motion or not motion_gate:
    objects = detect_objects(frame)
    if faces_enabled and (humans in objects or no object model):
        faces = recognize_faces(frame)          # deprecated, off by default
    threats = detect_threats(frame)             # not gated on a COCO person
collector.observe(all detections, frame)        # wrapped; never raises
return all detections
```

Two decisions worth knowing before changing this:

- **The motion gate saves CPU but never classifies a subject that holds still.** `MOTION_GATE=false` (`run_node.sh --every-frame`) runs YOLO on every frame, which is what requirement FR-19 asks for.
- **The threat stage is not gated on a COCO person box.** Gating it that way lost 20% of true positives on the 314-image test set (recall 0.800 to 0.612), because COCO never found those people.

`status()` reports what actually loaded (backend, model, gate, the share of frames where motion opened the gate, input size, whether threat detection is attached), not what was configured.

Face recognition (deprecated)

`src/detection/face_recognition.py` (`FaceRecognitionService`, ArcFace 512-d embeddings via InsightFace) and the Haar-cascade fallback remain in the code behind `FACE_RECOGNITION_ENABLED=false`. They were removed from the design on 2026-09-14: on CPU they took the pipeline from 23.4 to 0.86 FPS. The measured verification accuracy (0.9996 on LFW pairs, `scripts/benchmark_face_accuracy.py`) is still a valid result to report, with that cost stated alongside it.

3.7 `src/detection/annotate.py`: drawing the boxes

`draw_detections(frame, detections)` returns a **copy** with boxes and labels burned in; the clean frame is still needed for recording.

Colour (BGR)	Meaning
Orange `(0,165,255)`	`DANGEROUS_PERSON`
Red `(0,0,255)`	A face with no identity
Green `(0,255,0)`	Everything else

Motion boxes are never drawn; they cover most of the frame. When a threat box overlaps a COCO person box at IoU ≥ 0.5, only the threat box is drawn, so one person does not carry two stacked labels. The detection list itself is untouched.

Drawing happens server-side, before encoding, so a label can never drift out of sync with the frame it describes.

3.8 `src/detection/event_recorder.py`: clips around events

The camera runs 24/7; recording does not. `EventRecorder.process(frame, detections)` is called for every captured frame and decides on its own when to open and close a clip.

Behaviour	Default	Why
Triggers	person, dangerous person, face, vehicle, package (confidence ≥ 0.5)	Motion alone is excluded: clouds and branches would bring back 24/7 recording
Pre-roll	5 s ring buffer	A clip that starts at the detection has already missed the approach
Post-roll	10 s after the last trigger	Someone walking out of frame does not truncate the clip
Cooldown	continued detection extends the open clip	A loiterer gives one clip, not hundreds
Hard stop	120 s	A stuck trigger cannot fill the disk

Clips are written as `media/events/<camera>_<trigger>_<YYYYmmdd_HHMMSS>.mp4` (mp4v, 15 FPS). On close, `on_clip` puts a row in the `events` table (`event_type="clip"`), which is how the dashboard finds them. The recorder is fed the **clean** frame: saved footage is evidence, not a screenshot of the overlay.

Known gap: nothing deletes old clips. A long-running node fills its card.

3.9 Other detection files

File	What it is
`src/detection/models.py`	`ModelManager`: load YOLOv8n and get / set / diff its weights as numpy lists. Used only by the legacy YOLO-weights FL path (section 7)
`src/detection/benchmark.py`	A bare YOLO FPS loop from the early project. Use `scripts/benchmark_classes.py` or `sim/run_node.py` instead

3.10 Measured performance

Development laptop, four cores, pedestrian video:

Change	Result
Removing face recognition	0.86 → 23.4 FPS
Downscaled motion	17.8 → 22.9 FPS at 1080p
Smaller YOLO input	about 60% faster, about 5% fewer detections

On the Pi itself: the Hailo runs a large detector at 76 FPS through its own benchmark tool, and the NCNN export works. Per the 2026-09-27 status report, CPU-only on the Pi clears the revised 8-12 FPS target. **No end-to-end frame rate through ARGUS on the Hailo has been recorded**, because no `.hef` has been compiled yet. Do not quote laptop numbers as Pi numbers.

4 · Classification: the threat model

ARGUS classifies at two levels:

1. **What is it?** YOLOv8n on COCO, folded into person / vehicle / animal / package. Covered in section 3.
2. **Is this person dangerous?** A two-class YOLO11 model, covered here.
3. **Is this routine for this site?** The federated head, covered in section 5.

4.1 Where the model comes from

Source	`Nambekai/dangerous-person-detection-yolo11`, a graduation-thesis model by Tran Si Nam
Licence	AGPL-3.0. ARGUS is AGPL-3.0 as a result; see `NOTICE.md`
Classes	index 0 `nguoi_binh_thuong` (normal person), index 1 `doi_tuong_nguy_hiem` (dangerous person)
Variants	`yolo11n`, `yolo11s` (used), `yolo11s_preprocessing`, `yolo11m`, `yolo11l`
Weights	Not in git. `python scripts/fetch_models.py [--variant yolo11s]` downloads the upstream release into `models/` and verifies the published SHA-256
Default path	`models/threat-yolo11s.pt` (`THREAT_MODEL_PATH`)

The model is itself a person detector with two classes. It does not need COCO to find people first.

4.2 `src/detection/threat.py`

`ThreatClassifier(model_path, confidence=0.35, imgsz=416, crop_padding=0.15)`

The app constructs it with `THREAT_CONFIDENCE` (0.70) and `THREAT_IMGSZ` (416) from `Settings`, then attaches it with `detection_service.attach_threat_classifier()`. It raises a clear `RuntimeError` if `ultralytics` or the weights are missing; the app catches that and runs without the stage.

Classes are pinned by index, never by name. The upstream documentation calls them `normal_person` / `potentially_dangerous_person`, but the checkpoint ships Vietnamese names. Matching on the English strings silently detects nothing. At load time the class names are compared with `KNOWN_CLASS_NAMES` and a mismatch is logged at WARNING (INFO never reaches the demo unit's log, because uvicorn runs at warning level unless `DEBUG`).

Methods

Method	Used by	What it does
`detect_threats(frame)`	**The live cascade**	One inference over the whole frame; returns a `ThreatResult` with its own box for every class-1 detection
`classify_region(frame, bbox)`	Tests, tools	Pads the box by 15% of its longer side, crops, classifies
`classify_frame(frame)`	Tests, tools	Classifies the whole frame and reduces it to one verdict
`_classify(image)`	the two above	Any dangerous box beats every normal box, whatever the confidences

Why whole-frame, not per-person crops

Measured on the 314-image upstream test split:

Strategy	Precision	Recall	F1	Cost
Crop each COCO person	0.879	0.612	0.722	26.5 ms × number of people
Whole frame	**0.901**	**0.800**	**0.847**	35.6 ms flat

Per-crop lost 20% of true positives outright, because the COCO detector never found those people. It also gets slower exactly when a scene gets busy.

Why padding matters, when cropping is used

The model was trained on people in full scenes. A crop tight to the person box cuts off the arm and the weapon:

Crop	Result for the armed person in the upstream sample
Tight	no detection
15% padding	dangerous, 0.699
30% padding	dangerous, 0.533
50% padding	dangerous, 0.436

Why "any dangerous box wins"

On the upstream sample frame the bystander scores 0.86 normal and the armed person 0.62 dangerous. A plain argmax reports "nothing to see". Confidence therefore means "how sure we are about what we are reporting".

4.3 How the operating point was chosen

`scripts/sweep_threat_config.py` swept 48 configurations (2 variants × 4 input sizes × 6 thresholds) against the 314-image labelled test split plus 128 ordinary COCO images as negatives.

Read the baseline first. The test split is 250 positive / 64 negative, so a model that fires on every frame scores 0.796 plain accuracy. Plain accuracy is close to meaningless here; compare balanced accuracy, which is 0.500 for that always-fire baseline.

Configuration	Balanced accuracy	Precision	Recall	False alarms on ordinary scenes
yolo11n @ 416 @ 0.55 (old)	0.751	0.919	0.768	18.8%
yolo11s @ 416 @ 0.70 (current)	**0.773**	**0.926**	**0.796**	**14.8%**

The larger variant wins on every axis at once, including a lower false-alarm rate, and at a higher threshold. Nothing in the sweep reached 0.90 plain accuracy (best 0.847, which falsely flagged 53.9% of ordinary scenes). The defensible headline is precision: **when ARGUS flags someone, it is right more than nine times in ten.**

The tradeoff the threshold controls:

- A false negative misses an armed person, the failure the system exists to prevent.
- A false positive flags an innocent visitor as dangerous, in public, at a booth. That is a credibility and an ethical problem.

`scripts/tune_threat_threshold.py` prints precision, recall, F1 and both error counts at each threshold so the choice is made on evidence and written down.

4.4 Cost, and when to turn it on

Measured on x86 (Ryzen AI 7 350), YOLO11n, whole frame (`docs/THREAT_DETECTION.md`):

Input size	ms / frame	Note
320	about 16	misses entirely, do not use
416	**about 36**	the default: fastest usable, and most confident
512	28 to 45	
832 (trained size)	46 and up	no accuracy gain to justify it

On the Pi 5 CPU, `scripts/run_node.sh` budgets about 100 ms per frame.

That is why `scripts/run_node.sh` defaults it **off** and `--threat` turns it on. Once the detectors run on the Hailo, this constraint mostly goes away (`sim/run_node.py --profile pi5-threat-on-cpu` models the risky case where the threat model stays on the CPU).

4.5 Tools for this stage

Script	Use
`scripts/fetch_models.py`	Download and checksum the weights. `--list` shows variants
`scripts/sweep_threat_config.py --dataset <split> [--negatives <dir>] [--weights a.pt b.pt]`	Size × threshold sweep, optionally across variants
`scripts/tune_threat_threshold.py --dataset <split> [--negatives <dir>]`	Threshold-only sweep with both error counts
`scripts/benchmark_classes.py --images <dir>` or `--dataset <split>`	Per-class confidence spread for the whole cascade; with a dataset, true precision and recall for the threat stage

---

4.6 The tests

`tests/test_threat.py` (41 tests) covers class pinning by index, the mismatch warning, padding geometry at frame edges, "dangerous beats normal", whole-frame detection returning per-person boxes, the cascade running threats without a COCO person, a threat-stage exception not killing object detection, overlay colour, and the threat box superseding the overlapping person box. None of them load the real weights; the model is replaced by a stub, so they run without `models/`.

5 · Training and federated learning

This is the part of ARGUS that learns. YOLO and the threat model are frozen. What learns is a small head that decides whether a detection is routine for the site it was seen at, trained on that site's labelled events, and improved by averaging with other sites without sharing any footage.

5.1 The data path, end to end

```
process_frame detections
        │
EventCollector.observe          src/federated/collector.py   IoU + class tracking, one row per track
        │ track closes
features.extract(track)         src/federated/features.py    19 numbers
        │
SampleStore.append              src/federated/store.py       data/training/samples.jsonl  (label = None)
        │ a person confirms or corrects (dashboard, API, ingest_video.py)
SampleStore.set_label
        │
FederatedHead.fit               src/federated/head.py        local training on a copy
        │
node_client  ◄──── Flower round ────►  aggregator (robust)   src/federated/node_client.py
        │ global.npz
model_state.accept_candidate    src/federated/model_state.py the validation gate
        │ accepted
data/fl_head.npz  →  live head, version fl-rN
```

5.2 `features.py`: what the head sees

The head does not see pixels. It sees a **detection in context**, 19 numbers, all normalised into roughly [-1, 1] at the source so no scaler has to be fitted or shipped:

Index	Feature	How it is computed
0-3	class one-hot	person, vehicle, animal, package
4-5	width, height	box size as a fraction of the frame
6	sqrt(area)	
7	aspect	`min(w/h, 6) / 6`
8-9	centre x, centre y	fraction of the frame
10-11	velocity x, y	`tanh(10 × displacement)` between the last two updates
12	speed	`tanh(10 × |displacement|)`
13	dwell	seconds present / 30, capped at 1
14-15	hour of day	sin and cos, so 23:59 sits next to 00:00
16-17	day of week	sin and cos
18	detector confidence	

Five output labels: `routine_person` (0), `routine_vehicle` (1), `routine_animal` (2), `routine_package` (3), `anomaly` (4).

`class_index(detection_type, coco_id)` maps a detection onto the four classes. Packages are checked by COCO id first (24, 26, 28). Motion and unknown return `None` and are skipped rather than fed in as zeros.

`features.py` and `sim/workload.py` must stay in lockstep. The simulator generates vectors in this exact layout.

5.3 `collector.py`: one event per track, not per frame

`EventCollector.observe(detections, frame_shape, frame=...)` is called by `process_frame` on every frame.

- **Association.** Each detection joins the open track of the same class with the highest IoU above 0.3. If none overlaps, it falls back to centre distance within 2.5 box-widths, because a fast car can move further than its own width between detections and IoU alone would break it into one-frame tracks.
- **Closing.** A track unseen for 2 s closes. Tracks with fewer than 3 frames are dropped as detector flicker (`dropped_short`).
- **One row per track.** A person standing in frame for a minute is one training row, not 1,800. Otherwise the head learns that whatever loiters longest matters most.
- **Stills.** The still saved for an event is the frame where the subject was **largest** (updated only when the box grows 15%), not the last frame, by which time the subject is half out of shot. A crop of just the subject is saved alongside, so an image model can be trained later without re-recording. Both go to `data/snapshots/`.
- **Motion events.** Sustained motion over 0.2% of the frame for at least 1 s, with no classified object, becomes a `motion` event. It is how a person YOLO missed still leaves a record. Long motion is cut at 60 s.

- Every emitted row is also logged in the `events` table (`event_type="detection"`) with a human-readable feature summary.

5.4 `store.py` : the sample store

`SampleStore(path)` is an append-only JSONL file, `data/training/samples.jsonl` . One line per sample:

```
{"id": "18f2c3a9b1-00042", "n": 43, "t": 1727800000.0,
 "x": [0.0, 1.0, 0.0, 0.0, 0.31, ...], "y": null,
 "meta": {"snapshot": "...jpg", "crop": "...jpg", "track_id": 7, "frames": 41,
          "duration_s": 3.2, "source": "camera", "synthetic": false}}
```

`n` is the stable integer id the dashboard addresses. `y` is `null` until a person labels it. JSONL survives a restart, can be inspected with `tail` , and needs no migration when a field is added.

Why labels come from a person: the simulator derives labels from the same rule that generated its data, which is circular. On a real node the label has to come from outside the model.

`bootstrap(site, n)` seeds rows from the simulator's generator, tagged `synthetic: true` , so a demo can train before enough real events exist. `stats()` reports real and synthetic counts separately. `holdout_split()` is the one deterministic 80/20 split shared by the client and the gate, so a candidate is always scored on rows it did not train on.

5.5 `head.py` : the model

`FederatedHead` is a pure-numpy MLP:

```
19 → 384 → 384 → 128 → 5      ReLU, softmax, cross-entropy, Adam (lr 3e-3), batch 64
```

About 200K parameters, deliberately. Constraint CON-3 in the proposal caps the federated model there because training runs on the Pi 5 CPU at night. No torch import, so its memory use is honest, and Flower's `NumPyClient` wants lists of arrays anyway.

Method	
`fit(X, y, epochs, batch=64, seed)`	Mini-batch Adam; returns the last batch loss
`evaluate(X, y)`	(loss, accuracy)
`balanced_accuracy(X, y)`	Mean per-class recall. **Report this, never plain accuracy**: 254 of the first 322 real events were person, so always guessing person scores about 79%
`predict` , `infer_one`	Class probabilities / one label
`get_weights` , `set_weights` , `save` , `load`	Flower and `.npz` transport

`src/federated/head.py` mirrors `sim/head.py`. Keep them identical, or the simulator stops predicting what the node does.

5.6 `model_state.py`: the validation gate

Nothing reaches the live head without passing `accept_candidate(app, candidate_path, source, tolerance=0.01)`:

1. Score the live head and the candidate on this node's held-out 20%.
2. Reject if the candidate's balanced accuracy is more than `tolerance` below the live head's. If balanced accuracy is level (within `tolerance`), reject when the candidate's loss is more than `loss_tolerance` (5%, relative, with a 1e-4 floor) worse than the live head's. Otherwise accept. The very first model is accepted outright.
3. On acceptance, swap the weights in one assignment, save `data/fl_head.npz`, bump the version (`local-rN` or `fl-rN`).
4. Record every decision, accepted or not, in `data/fl_model_state.json` (last 50 kept).

`load_model_state(app)` restores the accepted head at startup, so an update survives a restart.

Every accepted version is also archived as `data/fl_models/<label>.npz` (newest 20 kept). `rollback(app, label)` makes an archived version live again as a **new** version, `rollback-rN`, recording what it restored and what it replaced; the counter never goes backwards. Exposed as `GET /api/federated/head/versions` and `POST /api/federated/head/rollback` (section 6.2). Operating it is covered in `docs/PRODUCTION_FL.md`.

Why the loss tie-break exists: balanced accuracy moves in coarse steps (0.25 with four represented classes), so two different models often score the same. In the 2026-09-20 hardware run, round 2's averaged model had worse loss (0.1941 → 0.2529) and was accepted at 0.75 against 0.75. Under the current gate that candidate is rejected, and the decision record carries the reason. A candidate that clearly improves balanced accuracy is still taken regardless of loss.

5.7 Local training: `POST /api/federated/head/train`

Implemented in `src/api/trial_routes.py`. Trains a **copy** of the live head on the labelled training split, off the event loop, then sends the copy through the gate with `source="local-train"`. Refuses with 409 below `min_samples` (200).

5.8 A federated round: `rounds.py` and `node_client.py`

`run_round_blocking(app, server, epochs=6)` is the one definition of a round. The cron and the manual trigger (`POST /api/federated/round`) both follow it so the 02:00 path cannot drift from the demo path.

1. Require 200 labelled samples.
2. Save the live weights to `data/fl_rounds/<timestamp>/init.npz`.
3. Launch `python -m src.federated.node_client --store ... --init ... --out ... --server ... --epochs ... --node-name ... [--central-api ...]` as its own process, with BLAS pinned to one thread (`ARGUS_TRAIN_THREADS`).

4. The client registers with the aggregator's REST API if `--central-api` is given, then joins the Flower round: * `fit()` loads the server's weights, trains on the node's labelled training split, returns weights plus real metrics (loss, accuracy, balanced accuracy, samples, seconds) and its `node_id`. * `evaluate()` receives the **aggregated** weights, scores them, and writes them to `global.npz`. After the last round that file holds the final global model.
5. Back in the API process, `global.npz` goes through the gate with `source="federated"`.

Everything lands in `data/fl_rounds/<timestamp>/` (`init.npz`, `global.npz`, `events.jsonl`), so any round can be inspected afterwards.

Only weights travel. No frames, stills or feature vectors leave the node.

5.9 `scheduler.py`: the cron

`FLScheduler` runs only when `FL_ENABLED=true`.

- Due every `FL_ROUND_INTERVAL_DAYS` (14) at `FL_ROUND_HOUR` (02:00).
- The last-round time is persisted to `data/fl_state.json`. Without that, a node rebooted weekly would restart a 14-day countdown forever and never train.
- A round overdue at startup runs after a 5-minute grace period, not a full interval later.
- It wakes at least hourly rather than sleeping for 14 days, which survives suspend, NTP clock steps on an RTC-less board, and lets `stop()` return promptly.
- A failed round does **not** advance the schedule, so it retries at the next wake. "Failed" means the round never happened: `run_round_blocking` returns `completed: false` when the aggregator is unreachable, the round times out, or there are too few labels, and the scheduler treats that as a failure. Until 2026-10-05 it did not: that result arrived as a dict, not an exception, so an unreachable aggregator was recorded as a finished round and the retry slipped 14 days. A round the gate rejected is `completed: true` and does advance the schedule.
- `is_idle()` defers a round if motion was seen in the last 10 minutes. **But nothing calls `notify_motion()`, so this check always passes today.**

5.10 `robust.py` and `robust_strategy.py`: Byzantine-robust aggregation

Plain FedAvg lets one broken or hostile node move the global model for everyone.
`RobustAggregator.aggregate(global_weights, updates)` screens **deltas** (client minus global), cheapest filter first:

Filter	Rejects	Defends against
1 Structural	wrong shapes, NaN/Inf, zero samples	a broken or mismatched node
2 Norm	update norm > 3× the median	scaled "model replacement" attacks, a diverged optimiser
3 Direction	cosine to the consensus direction < 0	sign-flip, label-flip, targeted poisoning
4 Trimmed mean	the top and bottom 10% per coordinate	residual outliers

Filters 3 and 4 need a majority to be honest, so they are skipped below 3 contributors and the report says so. `validate_candidate()` is the matching promotion check.

21 tests in `tests/test_robust_aggregation.py` cover the filters.

How it reaches the servers. `RobustFedAvg` in `src/federated/robust_strategy.py` is a Flower `FedAvg` subclass:

1. `configure_fit` remembers the parameters it sends out that round. They are the reference every returned update is measured against.
2. `aggregate_fit` turns each `FitRes` into a `ClientUpdate`, named by the `node_id` the client reports (falling back to Flower's connection id; duplicate names get a `#n` suffix so two nodes left on the default name are both screened), and calls `RobustAggregator.aggregate`.
3. Rejected nodes are logged at WARNING. Client metrics are averaged over the kept updates only. The round's metrics gain `robust_accepted`, `robust_rejected`, `robust_consensus` and `robust_report` (the full report as JSON).
4. If nothing survives, it returns no parameters, and Flower keeps the current global model. Nothing is checkpointed for that round.

`aggregation="fedavg"` skips all of this and calls the stock `FedAvg.aggregate_fit`: the baseline arm of the comparison study. One test asserts the switch reproduces stock FedAvg exactly; another runs the same sign-flip round through both arms and checks that only FedAvg is pulled off course (`tests/test_robust_strategy.py`, 11 tests).

Server	How to choose the arm
`central_server/fl_aggregator.py`	`"aggregation"` in `central_config.json`, `ARGUS_AGGREGATION`, or `run_central_server.ps1 -Aggregation fedavg`
`sim/fl.py serve`	`--aggregation robust` (default) or `fedavg`; each round's report is written to `sim/_run/fl_events.jsonl` as `kind: aggregation`
`src/federated/server.py`	`start_server(aggregation=...)`

Verified end to end on 2026-10-05: a real Flower server with three simulated clients over gRPC, two rounds, all three updates accepted each round with consensus filtering applied.

The honest limit is unchanged: with one or two nodes, filters 3 and 4 do not run, so on today's single Pi the screening can catch a broken or oversized update but not a disagreeing one.

5.11 The aggregator: `central_server/fl_aggregator.py`

One process, two jobs:

- **Flower server** on `ARGUS_FL_ADDRESS` (default `0.0.0.0:8080`). `ArgusStrategy` is `RobustFedAvg` (section 5.10) plus a hook that records per-client metrics and saves a checkpoint after every round. Each round's history entry carries the screening report (`aggregation`), and rejections count in the Prometheus counter `argus_fl_rejected_updates_total{node_id}`. A round where every update was rejected saves no checkpoint. The global version counter resumes from the highest `model_round_*_v<N>.npz` on disk at startup

(`resume_model_version`); before 2026-10-05 it restarted at 1 and a restarted server overwrote the previous run's checkpoints. Metrics are weighted by sample count (without that, Flower drops them and every dashboard reads zero). On start it resumes from the latest checkpoint.

- **Flask REST API** on 8090:

Route	
`GET /api/status`	server state
`GET /api/nodes` , `POST /api/nodes/register` , `POST /api/nodes/<id>/heartbeat`	node registry
`GET /api/training/history` , `/api/training/round/<n>` , `/api/training/summary`	per-round results
`GET /api/model/latest` , `/api/model/checkpoints`	global model versions
`GET/POST /api/config`	runtime config
`GET /metrics` , `/api/prometheus/targets`	Prometheus

Set `ARGUS_AUTH_TOKEN` to require a bearer token on the protected routes. Config comes from `central_config.json` overridden by `ARGUS_FL_ADDRESS` , `ARGUS_MIN_CLIENTS` , `ARGUS_ROUNDS` , `ARGUS_AGGREGATION` . `docker-compose.yml` runs it with Prometheus (9090), Grafana (3000) and a Cloudflare tunnel. The image builds from the repo root (`central_server/Dockerfile` , with its own `Dockerfile.dockerignore`) so it can copy `robust.py` and `robust_strategy.py` from `src/federated/` .

5.12 The simulator: `sim/`

File	Role
`workload.py`	Synthetic per-site events in the 19-feature layout. Each site has its own class mix, active hours, never-routine classes and loitering limit, so the data is non-IID in both label mix and the rule itself
`head.py`	Copy of the node's head
`fl.py`	`serve` (Flower server; robust aggregation, or `--aggregation fedavg`) and `client --site` (trains on generated data). Separate OS processes, one BLAS thread each
`pipeline.py`	The per-frame loop: `cpu` stages burn a real core, `accel` stages sleep (the Hailo works over PCIe while the CPU is free), `measured` stages time the real head
`profiles.py`	Stage timings, each with its source
`run_node.py`	One node at a target FPS with a live Flower round against it; reports the FPS hit, Prometheus on 9108

The three sites:

Site	Class mix (person/vehicle/animal/package)	Active hours	Rule
`driveway`	40 / 42 / 10 / 8	06-22	Night presence is the signal
`backdoor`	46 / 4 / 14 / 36	08-19	A vehicle here is always wrong
`yard`	22 / 3 / 56 / 19	07-20	Tolerant of loitering, not of people at night

Profiles:

Profile	CPU ms	Accelerator ms	Serial ceiling	Pipelined
`pi5-hailo` (target build, default)	27	40	14.9 FPS	25.0 FPS
`pi5-threat-on-cpu` (risk)	63	28	11.0 FPS	15.9 FPS
`pi5-cpu-only` (fallback)	405	0	2.5 FPS	2.5 FPS
`x86-dev` (measured)	140	0	7.1 FPS	7.1 FPS

What it answered: the sponsor spec rejected every 8 GB board on the claim that a node "stops detecting while it trains". That claim came from Jetson unified memory. With detection on the Hailo and a 200K-parameter head on the CPU, the simulator measured a round costing about 0.1% of frame rate.

What it cannot answer: whether the head is accurate on real data, because the team wrote both the generator and the model.

5.13 Results so far

Run	Setup	Result
Hardware, 2026-09-20	One Pi, laptop aggregator, 2 rounds, 322 hand-labelled events	Balanced accuracy 0.75. `routine_vehicle` had 0 samples and `routine_animal` 1, so only three of five classes were represented
Simulation, 2026-10-02	3 synthetic sites, 3 rounds, `sim/fl.py`	Balanced accuracy 0.820 → 0.829 → 0.838

Neither is yet evidence that federated beats local-only training. That needs the three-arm comparison (untrained / local-only / federated, on one frozen holdout, ideally leave-one-site-out) in the 2026-09-27 plan.

The fine-taxonomy study (2026-10-03, section 5.14) is the first result that bears on that question, and it points the other way: in every run, local-only training beat plain FedAvg on site-specific vehicle makes.

5.14 Datasets: the CIFAR-10 stand-in and what replaced it

The project trained on four kinds of data, in this order. Only the last two exercise the head ARGUS actually runs.

Stage	Data	What it proved	Files
1. Plumbing, April	CIFAR-10, 2 nodes, **IID** split (`IidPartitioner`)	Flower works end to end; a CNN reached 88.86% eval accuracy. Validates plumbing, not learning: an IID split is the easiest case FL has	`Dropbox/argus/federated-learning/flower-federated-learning-cifar10/` (`tfexample/` code, `TRAINING_REPORT.md`); the same report as `Dropbox/argus/argus/flower_training_report_2026-04-24.md`
1b. Early sandboxes	MNIST (Keras); DETR on CPPE-5; centralised vs local vs federated on synthetic data	Ideas only	`Dropbox/argus/argus/federated_learning_keras.py` ; `Dropbox/argus/federated-learning/federated-learning-sandbox/` (`flower_detr/` , `sandbox.py`)
2. The ARGUS workload, September	Synthetic per-site events in the head's 19-feature layout, three sites, **non-IID** in both label mix and the rule itself	The federated head trains and averages across dissimilar sites. Cannot prove real-world accuracy: the team wrote the generator and the model	`sim/workload.py` , `sim/fl.py` , `sim/README.md` (repo)
3. Real events, 2026-09-20	322 hand-labelled events from the Pi, `data/training/samples.jsonl` on the node	Two real rounds Pi ↔ laptop. Balanced accuracy pinned at 0.75 by two near-	`scripts/ingest_video.py` (now with `--label-by-class` to fill vehicle and animal from street footage), section 5.13

Stage	Data	What it proved	Files
		empty classes	
4. Fine taxonomy, 2026-10-03	Stanford Cars (16,185 photos, 49 makes) for a crop classifier whose outputs add 16 features to the head (19 → 35)	Make classifier 0.832 test accuracy; in simulation FedAvg balanced accuracy 0.860 → 0.894 with it. Local-only beat FedAvg in every run	`Dropbox/argus/argus/ARGUS_Fine_Taxonomy_2026-10-03_v2.pdf`. Code is on branch `feat/fine-taxonomy` in the `argus-fl` working copy on the other development machine, not on GitHub yet; data under `D:\argus-data` on that machine

Why CIFAR-10 was retired: `sim/workload.py` says it directly. The April report split CIFAR-10 with `IidPartitioner`, "each node gets a random, balanced sample", which is the one case where federation has nothing to overcome. The head also does not classify pixels: YOLO already says "vehicle", and the head's job is whether that vehicle is routine *here*.

CIFAR-10 comes back once, deliberately, in the 135-run study (section 10.5): with **Dirichlet** partitioning, alongside FEMNIST, so the robust aggregation result can be compared with published FL numbers. It is never to be split IID again.

6 · API, data and security

The node is one FastAPI app (`src/api/app.py` , `create_app()`), served by uvicorn from `main.py` on port 8000. Interactive docs are at `http://<node>:8000/docs` .

6.1 Startup order (`src/api/app.py` , `startup_event`)

The order matters; each step depends on the ones before it.

1. **Database**: `Database(settings.DB_PATH).initialize()` .
2. **Auth**: `initialize_auth(db)` creates the admin account if missing (from `ARGUS_ADMIN_PASSWORD` , or a generated password printed **once** at WARNING level) and warns if `AUTH_REQUIRED` is off or `SECRET_KEY` is the default.
3. **Cameras**: build the registry from `CAMERA_SENSORS` ; configure `camera_service` from settings. Nothing is opened yet.
4. **Detection**: build `DetectionConfig` from settings and `detection_service.initialize()` (picks the backend).
5. **Face recognition**: only if `FACE_RECOGNITION_ENABLED` (deprecated).
6. **Threat classifier**: only if `THREAT_ENABLED` ; failure is a warning, not a crash.
7. **Event recorder**: only if `RECORD_EVENTS` ; finished clips are logged to the `events` table.
8. **Federated head, sample store, collector**: always, regardless of `FL_ENABLED` , because capturing and labelling is useful before any round is scheduled. Then restore the last accepted head.
9. **Detection worker**: started if `DETECTION_AUTOSTART` (default true), so detection runs with nobody watching.
10. **FL scheduler**: only if `FL_ENABLED` .

Every optional stage is wrapped so its failure leaves the rest running: losing training data or the threat stage is recoverable; losing the camera during a demo is not.

6.2 Routes

`P` marks routes behind `require_auth` , enforced only when `AUTH_REQUIRED=true` . Routes without P are open in every mode, including all federated and labelling routes (see 6.4).

Health and the built-in page (`routes.py`)

Method	Path	
GET	`/`	Service banner
GET	`/dashboard`	`static/index.html` , the built-in live page
GET	`/api/status`	`{fps, accelerator, model_version, nodes_online, node, samples}`
GET	`/api/detection/status`	What actually loaded: backend, model, gate, motion rate, worker and collector stats

Camera (`routes.py`)

Method	Path	
GET	`/api/camera/status`	
POST P	`/api/camera/init` , `/api/camera/shutdown`	
GET P	`/api/camera/snapshot`	One JPEG
GET P	`/api/camera/stream`	MJPEG with boxes; `?token=` accepted because an `<img>` cannot send headers
POST P	`/api/camera/record`	Manual recording
GET	`/api/cameras` , `/api/cameras/{name}/status`	Multi-camera registry
POST P	`/api/cameras/init` , `/api/cameras/{name}/init` , `/api/cameras/{name}/shutdown`	
GET P	`/api/cameras/{name}/snapshot` , `/api/cameras/{name}/stream`	
GET P	`/api/recordings` , `/api/clips`	Saved footage

Events and labelling (`routes.py` , `dashboard_routes.py`)

Method	Path	
GET	`/api/events` , `/api/events/{id}`	Real events from the sample store, in the dashboard's `DetectionEvent` shape; synthetic rows excluded
GET P	`/api/events/{id}/snapshot`	The still saved for an event
POST	`/api/events/{n}/label`	`{"action": "confirm"}` or `{"action": "correct", "label": "<free text>"}`
WS	`/ws/events`	Pushes each new real event, polling the store once a second
POST	`/api/login`	Dashboard login, returns `{access_token}`

How a dashboard label becomes a class (`_resolve_label`):

- **confirm** on a flagged event means `anomaly` ; on anything else it means that class is routine here.
- **correct** with no text inverts the call.
- **correct** with text reads keywords: "stranger", "suspicious", "intruder", "threat" and similar mean `anomaly` ; "dog", "van", "courier" and similar pick the class; "routine", "neighbour", "false alarm" mean routine for the event's class. Free text never 422s, because the dashboard's Correct box is free text.

Federated (`trial_routes.py`, `routes.py`)

Method	Path	
GET	`/api/federated/status`	Scheduler state, last and next round
GET	`/api/federated/samples`	Recent samples and counts
POST	`/api/federated/samples/{id}/label`	`{"label": 0..4}`
POST	`/api/federated/samples/bootstrap`	`{"site", "n", "seed"}`, synthetic rows
GET	`/api/federated/head`	Head size, version, gate history
GET	`/api/federated/head/versions`	Archived versions, newest first, and which is live
POST	`/api/federated/head/rollback`	`{"label": "fl-r3"}`, make an archived version live as a new version (409 during a manual round, 404 if not archived)
POST	`/api/federated/head/train`	`{"epochs", "min_samples", "tolerance"}`, train a copy and gate it
POST	`/api/federated/predict`	`{"features": [19 numbers]}`
POST	`/api/federated/round`	`{"server", "epochs", "min_samples", "tolerance"}`, start a round (409 if one is running)
GET	`/api/federated/round`	`{running, result, log}` of the current or last round

Auth and logs (`routes.py`)

Method	Path	
POST	`/api/auth/login`, `/api/auth/logout`	Bearer tokens
GET	`/api/auth/me`	Includes `auth_required`, which `run_tunnel.sh` checks
GET P	`/api/logs`	Audit log

Legacy and dev routes

Path	Status
`/api/faces` (GET, POST, DELETE `{id}`, POST `reset`)	Face enrollment. Works, but face recognition is deprecated and off
`/api/devices`, `/api/automations`	Backed by the in-memory `AutomationService` stub; not used by the demo
`/api/errors/analyze`, `/api/errors/fix`, `/api/errors/history`, `/api/agents/status`	The error-routing helper agents (section 7)

6.3 The database (`src/database/__init__.py`)

SQLite at `DB_PATH` (default `data/database.db`). `ARGUS_DATA_DIR` moves the database, the sample store, the head and the FL state together, which is how `mock_demo.py` isolates itself in `data/mock` .

Table	Holds
`events`	Every emitted detection (`event_type="detection"`) and every saved clip (`"clip"`), with a JSON `data` column and `media_path`
`users`	Accounts: id, username, PBKDF2 hash, role
`audit_log`	Security-relevant actions
`faces`	Face embeddings (deprecated path)
`devices` , `automations`	Automation stub

Training data does **not** live in SQLite. It lives in `data/training/samples.jsonl` (section 5.4).

Everything on disk

Path	What	Safe to delete?
`data/database.db`	events, users, audit	Loses accounts and history
`data/training/samples.jsonl`	every sample and its label	**Loses all labelling work**
`data/snapshots/`	event stills and crops	Events lose their pictures
`data/fl_head.npz`	the accepted head	Node reverts to an untrained head
`data/fl_model_state.json`	version and gate history	Version counter restarts
`data/fl_state.json`	last round time	Next round is treated as overdue
`data/fl_rounds/<ts>/`	per-round artefacts	Yes
`media/events/*.mp4`	event clips	Yes, and **nothing deletes these automatically**
`models/`	exports and threat weights	Rebuild with the export and fetch scripts
`central_server/checkpoints/`	global model per round	On the aggregator only

6.4 Security (`src/security/__init__.py`, `src/api/auth.py`)

Piece	Implementation
Password hashing	PBKDF2-HMAC-SHA256, 100,000 iterations, 16-byte random salt
Tokens	`secrets.token_urlsafe(32)`, 24 h expiry, held **in memory**: a restart logs everyone out
Enforcement	`require_auth` FastAPI dependency on the routes marked P; `Authorization: Bearer` or `?token=`
Off by default	`AUTH_REQUIRED=false` so the LAN demo works without login. Acceptable behind a router only
Admin bootstrap	Created on first start; `scripts/set_password.py` resets a password or creates users
Remote access	`scripts/run_tunnel.sh` refuses to start unless auth is on and proven to return 401
Encryption at rest	`encrypt` / `decrypt` are **placeholders** (TODO AES-256); nothing is encrypted at rest yet

Known gaps, both to fix before anything is exposed:

- **The federated and dashboard routers carry no auth at all.** Only routes in `routes.py` marked P use `require_auth`. Everything in `trial_routes.py` (`/api/federated/*`: label, bootstrap, train, predict, round) and in `dashboard_routes.py` (`POST /api/events/{n}/label`, `WS /ws/events`) is open **even with** `AUTH_REQUIRED=true`. Anyone who can reach the port can relabel training data, seed synthetic rows, or start a round. `run_tunnel.sh` only probes `/api/faces`, so its preflight passes regardless. Fix: add `dependencies=[Depends(require_auth)]` to those two `APIRouter()`s and probe one of their routes in the tunnel preflight.
- The Next.js dashboard does not send its token to the backend.

6.5 The built-in dashboard (`static/index.html`)

One HTML file served at `/dashboard`, no build step. It shows the live annotated stream, detection FPS each second, what is currently in frame, and the event list with each event's still. It talks only to the node it is served from, so it works whenever the node does, even when the Next.js dashboard cannot reach it.

The product dashboard, `adam9798/argus-dashboard` (Next.js), is built against the contract implemented in `dashboard_routes.py`: `POST /api/login`, `GET /api/status`, `GET /api/events`, `POST /api/events/{id}/label`, `WS /ws/events`.

7 · Scripts, tools and legacy code

7.1 Runners (`scripts/`)

Script	Machine	What it does
`run_node.sh`	PI	Start the node with demo defaults. Flags: `--every-frame`, `--threat`, `--model`, `--video`, `--no-record`, `--port`
`node_status.sh [--once]`	PI	One status line per second from `/api/detection/status` and `/api/status`
`fl_trial.sh [--rounds N] [--epochs N] [--samples N] [--no-bootstrap]`	PI	Local aggregator + seed + local train + one round + gate decision
`run_central_server.ps1 [-Rounds] [-MinClients] [-FlPort] [-ApiPort] [-Token]`	LAPTOP	The aggregator, kept up between sessions; prints the lines to paste on the Pi
`run_dashboard.ps1 -Node <ip>`	LAPTOP	Clone, install and start the Next.js dashboard pointed at a node
`run_tunnel.sh [--check]`	PI	Cloudflare tunnel, refused unless auth is on and proven
`mock_demo.py [--reset] [--source] [--threat] [--record]`	ANY	The full node against a video, data isolated in `data/mock`

7.2 Data and labelling

Script	What it does
`ingest_video.py <video> [--label L] [--every-frame] [--max-frames N] [--zone Z] [--data-dir D]`	Replays a recording through the real pipeline and labels every event. Uses the video's own clock, so speed and dwell are right even when it replays faster than real time. Stop the node first
`check_classes.py --camera / <video> / <image> [--save out.jpg]`	Shows ARGUS class and raw COCO label side by side
`set_password.py [--user U] [--create --role R]`	Reset or create an account

7.3 Models: fetch, export, accelerate

Script	Runs on	What it does
`fetch_models.py [--variant] [--list]`	ANY	Download threat weights, verify SHA-256
`export_cpu_models.py --format ncnn / openvino`	the target	CPU-optimised YOLO export into `models/`
`export_hailo.py --hw-arch hailo8 / hailo10h [--check]`	**x86_64 Linux only**	Hailo export directory. Refuses on the wrong platform rather than failing halfway through a long build
`export_tensorrt.py [--model object] [--no-half] [--check]`	**the Jetson itself**	TensorRT engines; not portable across devices or JetPack versions

7.4 Measurement

Script	What it measures
`benchmark_classes.py --images <dir>` or `--dataset <split> [--limit N]`	Per-class confidence: mean, median, 5th percentile, spread, share within 0.10 of the threshold. With a dataset, threat precision and recall
`sweep_threat_config.py --dataset <split> [--negatives <dir>] [--weights ...]`	Threat input size × threshold, across variants
`tune_threat_threshold.py --dataset <split> [--negatives <dir>]`	Threat threshold only, both error counts
`benchmark_face_accuracy.py --faces <dir>`	Face verification accuracy and TAR@FAR (deprecated stage)
`verify_enrollment.py --name N [--keep]`	Live face enrollment on the Pi (deprecated stage)

7.5 Documentation

Script	What it does
`build_docs_pdf.py --master [--out-dir D]`	One PDF: this reference, the eight guides, the quick start, the simulator notes
`build_docs_pdf.py --guides`	The eight guides, one PDF each
`build_docs_pdf.py [--docs NAME ...]`	Selected docs from `docs/`, one PDF each

Uses headless Chrome rather than a print library (weasyprint needs GTK on Windows; wkhtmltopdf is unmaintained). No web fonts, so an offline machine renders the same tables.

7.6 The simulator (`sim/`)

Described in section 5.12. Entry points: `sim/fl.py serve|client` and `sim/run_node.py`. Its own notes are in `sim/README.md`, included at the end of the master PDF.

7.7 Legacy code: present, tested in places, not on the running path

These files are from the earlier design, where federated learning meant fine-tuning YOLO's own weights on each Pi. That design was replaced by the 19-feature head: YOLOv8n is 3.2M parameters against the head's ~200K (constraint CON-3), and the node runs YOLO from a compiled export, so averaged PyTorch weights could not reach the Hailo without recompiling the `.hef` on an x86 machine after every round.

File	Was	Now
`run_client.py`	Started a YOLO-weights Flower client	Not used. The node uses `src/federated/node_client.py`
`src/federated/client.py`	`ArgusFlowerClient` exchanging YOLO weights	Not used by the node; `FLScheduler` falls back to it only if no runner is given
`src/federated/server.py`	Minimal Flower server	Not used
`src/federated/utils.py`	Compress / delta helpers for weight lists	Not used by the running path
`src/training/__init__.py`, `dataset.py`	`LocalTrainer`, `LocalDetectionDataset`: YOLO crops on disk, export to YOLO format, `model.train()`	Not used
`src/detection/models.py`	`ModelManager`: YOLO weight get / set / delta	Used only by the files above
`server/app.py`, `server/config.py`, `server/strategy.py`	FedAvg / FedProx server driven by `STRATEGY`, `NUM_ROUNDS`, `MIN_CLIENTS`, `FL_PORT`	Not used by the node; `central_server/` replaced it
`src/detection/stream.py`	Detect-inside-the-stream MJPEG generator	Replaced by `stream_from_worker`; kept for tests
`src/detection/face_recognition.py`	ArcFace identity	Deprecated 2026-09-14, off by default
`src/detection/benchmark.py`	Bare YOLO FPS loop	Superseded by the scripts and the simulator
`src/automation/__init__.py`	Devices and rules (lights, locks, GPIO)	Stub behind `/api/devices`, `/api/automations`

Do not delete them casually: some are still imported by tests, and the proposal describes parts of them. Do not extend them either.

7.8 Research prototypes (`research/`)

Standalone experiments, not imported by the node.

File	What it is
`wifi_csi.py`	Presence and motion from WiFi channel state information (ESP32 or Nexmon backends, plus a mock), with an experimental pose estimator
`sensor_fusion.py`	Merges WiFi CSI presence with camera YOLO detections (Bayesian confidence merging; WiFi can wake the camera, the camera can veto WiFi false positives)
`argus_detect.py`	Early Hailo detection callback with PIR sensor and software motion
`federated.py`	First federated prototype with Prometheus metrics and node registration; the ideas moved into `central_server/`
`fl_config.json` , `csi_config.json`	Their configs

7.9 Developer tooling inside the app

Package	What it is
`src/agents/`	`ErrorRouter` classifies an error report and routes it to `SyntaxAgent` , `DependencyAgent` , `HardwareAgent` or `NetworkAgent` , which suggest or apply a fix; `FederatedAgent` is a separate early helper. Exposed at `/api/errors/*` and `/api/agents/status` . A development aid, not part of detection
`prometheus/`	**Not** the metrics system. A separate voice-activated orchestrator ("Hey Prometheus", "switch to argus") used during development; `python -m prometheus.main` . Documented in `docs/PROMETHEUS.md`

The Prometheus *metrics* system (the time-series database scraping `/metrics`) is a different thing entirely and lives in `central_server/prometheus.yml` .

8 · Configuration reference

All node settings live in `src/config.py` (`Settings`) and are read from environment variables once, at import, by `Settings.from_env()`. Booleans are the string `true` or `false`. There is no config file for the node.

`scripts/run_node.sh` overrides several defaults; its value is shown where it differs.

8.1 Server

Variable	Default	
`HOST`	`0.0.0.0`	Bind address
`PORT`	`8000`	
`DEBUG`	`false`	`true` turns on uvicorn reload and INFO logging. Otherwise uvicorn logs at WARNING, which is why important problems are logged as warnings
`SECRET_KEY`	`dev-secret-change-in-production`	Set a real value anywhere beyond a bench
`CORS_ORIGINS`	empty (`run_node.sh`: localhost and 127.0.0.1 on 3000 and 3001)	Comma-separated. Only needed for a separately hosted frontend
`ARGUS_DATA_DIR`	`data`	Moves the database, samples, head and FL state together. Relative paths are from the repo root

8.2 Camera

Variable	Default	
`CAMERA_INDEX`	`0`	Sensor for the single-camera service
`CAMERA_SENSORS`	empty	`"0:front,1:back"` for several cameras (Jetson Orin)
`CAMERA_SOURCE`	empty	A video file or numeric index; overrides the board camera
`CAMERA_SOURCE_REALTIME`	`true`	Pace a file at its own FPS; `false` reads as fast as possible, for benchmarks
`CAMERA_SWAP_RB`	`false`	Only if people look blue

Resolution (1920×1080), camera FPS (30) and rotation (0) are dataclass defaults in `Settings` with no environment variable.

8.3 Detection

Variable	Default	
`OBJECT_MODEL`	`yolov8n`	`yolov8s`, `yolov8m` ... An accelerated backend needs an export with the same name
`OBJECT_BACKEND`	`auto`	`auto`, `pt`, `openvino`, `ncnn`. Accelerators (TensorRT, Hailo) are always preferred when their files exist
`OBJECT_IMGSZ`	`640`	Ignored, with a warning, when an export fixes its own size
`DETECTION_THRESHOLD`	`0.5`	YOLO confidence
`MOTION_GATE`	`true`	`false` runs YOLO and threats on every frame
`MOTION_SENSITIVITY`	`25`	Pixel-difference threshold
`MOTION_WIDTH`	`320`	Width motion detection works at
`DETECTION_AUTOSTART`	`true`	Run the background worker from startup
`STREAM_MAX_FPS`	`15`	Viewer stream cap
`STREAM_WIDTH`	`960`	Viewer stream encode width

8.4 Threat stage

Variable	Default	
`THREAT_ENABLED`	`true` (`run_node.sh`: `false`, `--threat` turns it on)	
`THREAT_MODEL_PATH`	`models/threat-yolo11s.pt`	
`THREAT_CONFIDENCE`	`0.70`	Chosen by sweep, section 4.3
`THREAT_IMGSZ`	`416`	

8.5 Recording

Variable	Default	
`RECORD_EVENTS`	`true`	
`RECORD_PRE_ROLL_S`	`5`	
`RECORD_POST_ROLL_S`	`10`	
`RECORD_MAX_CLIP_S`	`120`	

8.6 Federated learning (node)

Variable	Default	
`NODE_NAME`	`Node A`	Shown on the dashboard and sent to the aggregator as `node_id`
`CAMERA_ZONE`	`Front door`	
`FL_ENABLED`	`false`	Turns on the scheduler. Capture, labelling and manual rounds work without it
`FL_SERVER_URL`	`localhost:8080`	Flower gRPC address of the aggregator
`FL_CENTRAL_API`	empty	Aggregator REST, e.g. `http://192.168.1.50:8090`; enables node registration
`FL_ROUND_INTERVAL_DAYS`	`14`	
`FL_ROUND_HOUR`	`2`	
`FL_LOCAL_EPOCHS`	`5`	
`FL_MIN_SAMPLES`	`50`	
`ARGUS_TRAIN_THREADS`	`1`	BLAS threads for the training subprocess

8.7 Security

Variable	Default	
`AUTH_REQUIRED`	`false`	Enforce bearer tokens on protected routes. See the gap in section 6.4
`ARGUS_ADMIN_PASSWORD`	unset	Used only when the admin account is first created; otherwise a password is generated and printed once

8.8 Deprecated

Variable	Default	
`FACE_RECOGNITION_ENABLED`	`false`	
`FACE_RECOGNITION_THRESHOLD`	`0.6`	
`FACE_SIMILARITY_THRESHOLD`	`0.4`	

8.9 The aggregator (`central_server/fl_aggregator.py`)

Settings are layered: built-in defaults, then the config file `central_server/central_config.json` (or the path in `ARGUS_CONFIG`), then environment variables.

Setting	Code default	Shipped config file	Environment override
Flower address	`0.0.0.0:8080`	`0.0.0.0:8080`	`ARGUS_FL_ADDRESS`
REST port	8090	8090	none
Clients per round	2	**1**	`ARGUS_MIN_CLIENTS` (`run_central_server.ps1 -MinClients`, default 1)
Rounds per session	50	**5**	`ARGUS_ROUNDS` (`-Rounds`, default 5)
Local epochs sent to clients	3	**6**	none
Auth token	empty	empty	`ARGUS_AUTH_TOKEN` (`-Token`)
Aggregation	`robust`	`robust`	`ARGUS_AGGREGATION` (`-Aggregation robust` or `fedavg`)

`ARGUS_SERVE_FOREVER` (default `true`) starts a new session when one finishes, so a node on a 14-day schedule still finds the server.

With one client per round the server will happily "federate" a single node. Set `-MinClients 2` or more when two nodes are actually running.

8.10 The legacy server (`server/config.py`)

`FL_PORT`, `NUM_ROUNDS`, `MIN_CLIENTS`, `STRATEGY` (`fedavg` or `fedprox`), `FEDPROX_MU`. Not used by the current node.

9 · Tests

9.1 Running them

```
python -m pytest -q                       # everything
python -m pytest -q -m "not slow"         # skip process spawns, video decode, network timeouts
python -m pytest tests/test_threat.py -q  # one file
python -m pytest -q -k "gate"             # by name
```

Last full run: 2026-10-05, branch `feat/threat-detection` at `a5f2c39` plus the working-tree changes in section 10.4, Windows 11, CPU-only torch: 328 passed, 0 failed, 0 skipped, 56 s.

Prerequisites, all in `requirements.txt` and all needed for a clean run:

Missing	Symptom
`pytest-asyncio`	4 failures in `test_fl_schedule.py::TestRoundOutcome` ("async def functions are not natively supported")
`ultralytics`	`test_a_clip_becomes_labelled_events` fails: the ingest finds 0 events because no detector loads
`media/vtest.avi`	`test_a_clip_becomes_labelled_events` is skipped, and `test_an_unknown_label_is_rejected` **fails** (it checks the video before the label)

Fetch the video with the `curl` line in section 2.1.

Gate any push on the exit code (`$?` is 0), never on reading the output. A grep for "failed" also matches a passing summary line, which is how two failing tests were once pushed.

9.2 How the suite stays fast and hermetic

`tests/conftest.py` sets two variables before anything imports `src.config`:

- `DETECTION_AUTOSTART=false`, so no test grabs the developer's webcam.
- `OBJECT_BACKEND=pt`, so an export lying in `models/` cannot change which backend a test sees.

`pytest.ini` runs asyncio in strict mode, so an async test missing its marker fails loudly instead of passing while asserting nothing. Models are replaced by stubs almost everywhere, so most tests need no weights.

9.3 What each file covers

File	Tests	Covers
`test_pipeline_changes.py`	50	The September changes, each pinned: face stage deprecated, downscaled motion, the detection worker, file camera source, **the model gate** (first model accepted and persisted, regressing candidate rejected, loss breaks a balanced-accuracy tie, a clear accuracy gain is taken despite worse loss, head restored at startup), dashboard corrections to labels, motion segments, head weight isolation, the motion gate, snapshots while the worker owns the camera, camera colour order, the ARGUS class taxonomy, model selection, stills and clips, fixed-shape exports
`test_threat.py`	41	Class index pinning, construction errors, whole-frame threat detection, crop padding and clamping, the cascade stage, status, overlay colour and dedup
`test_jetson.py`	35	Board detection from the device tree, backend selection, the GStreamer CSI pipeline, the camera registry, Hailo export probing, camera state
`test_auth.py`	33	Login, protected endpoints refuse without a token and accept with one, open endpoints stay open, logout, auth disabled mode, PBKDF2 hashing
`test_fl_schedule.py`	26	Interval and due-time arithmetic, persistence across restarts, the idle gate, status, round outcomes (failure does not consume the interval, an unreachable aggregator through the real runner is retried, a gate rejection still counts as a round, success advances it, disabled FL does not run)
`test_robust_aggregation.py`	21	Structural, norm and direction defences, the consensus floor below 3 clients, aggregation outcome, `validate_candidate`
`test_robust_strategy.py`	11	The Flower strategy: a sign-flip attacker is left out of the round's model, the `fedavg` switch reproduces stock FedAvg, the same attack moves FedAvg but not robust, nothing surviving keeps the current model, the reference is the model sent out, duplicate node names, metrics averaged over kept updates, a failing report callback, the central server reading `ARGUS_AGGREGATION`
`test_model_versions.py`	8	Accepted versions archived and rejected ones not, rollback restores weights as a new version and survives restart, unknown or path-like labels refused, archive pruning, the versions and rollback routes, the aggregator's version counter resuming and never moving backwards
`test_face_recognition.py`	20	Deprecated face path: reset, enroll, recognise, remove, availability
`test_event_recorder.py`	18	Triggers, pre-roll, post-roll and cooldown, output files
`test_annotate.py`	11	Box drawing and label text
`test_api.py`	11	Health, camera, detection, automation and security endpoints respond
`test_dashboard.py`	11	`/dashboard` is served and contains what the page needs
`test_detection_service.py`	11	Initialisation, cascade fallbacks, status
`test_stream_annotated.py`	9	MJPEG framing, detection cadence, resilience to a failing detector

File	Tests	Covers
`test_federated_infra.py`	6	Round guards (unreachable aggregator fails cleanly), video ingest end to end, scheduler lifecycle
`test_face_api.py`	4	Face reset endpoint, detection status
`test_stream_endpoint.py`	2	The stream route

Slowest tests (each 8 to 14 s): the three `TestModelGate` cases (app startup), the video ingest, and the unreachable-aggregator timeout.

9.4 What is not tested

- Real hardware: Picamera2, the Hailo, the Jetson CSI path. Covered by stubs; verified by hand on the Pi.
- `central_server/fl_aggregator.py` has three tests (its aggregation setting and the version counter); the strategy it inherits is tested in `test_robust_strategy.py`. The REST API and checkpointing are exercised by `scripts/fl_trial.sh` and the hardware runs.
- `sim/` has no unit tests. It is exercised by running it.
- Auth on the federated and dashboard routers, because there is none (section 6.4). A test that those routes return 401 with `AUTH_REQUIRED=true` should be written together with the fix.
- No coverage report is configured. Add `pytest-cov` and `--cov=src --cov-report=term-missing` to measure it.

10 · Status, known gaps, and what to do next

As of 2026-10-05. Showcase 2026-11-22; feature freeze 2026-11-08.

10.1 What works and has been run

Capability	Evidence
Camera → motion → YOLO → overlay → live stream, continuous and unattended	First end-to-end hardware run on the Pi, 2026-09-20
Event tracking, one event per track, stills and crops	322 hand-labelled real events from that run
Event-triggered clips with pre- and post-roll	`media/events/` on the Pi; 18 tests
Threat stage	0.926 precision, 0.796 recall at the chosen operating point (section 4.3)
Local head training through the gate	API and tests
Federated round Pi ↔ laptop aggregator	2 rounds on 2026-09-20, balanced accuracy 0.75
Multi-client Flower simulation	3 clients × 3 rounds on 2026-10-02, section 2.7
Robust aggregation in the Flower servers	Wired 2026-10-05; 3 clients × 2 rounds over gRPC through `RobustFedAvg`; 11 strategy tests (section 5.10)
Gate loss tie-break	2 tests; the 2026-09-20 round-2 case is now rejected (section 5.6)
Fine-taxonomy make classifier	0.832 over 49 makes on Stanford Cars; FedAvg balanced accuracy 0.860 → 0.894 in simulation (section 5.14)
Model versions and rollback	Node archive, rollback API, aggregator version resume; 8 tests (sections 5.6, 5.11)
Cron retries a round that never happened	2 tests through the real runner path (section 5.9)
Test suite	328 / 328 on 2026-10-05

10.2 Gaps that block the research claim

Gap	Why it matters	Fix
One physical node	FedAvg over one client averages it with itself. That is not federation	Second Pi 5 + AI HAT+ 2. Meanwhile, simulation with 3 or more clients is the primary evidence
Robust filters idle below 3 nodes	Wired in, but with one Pi only the structural and norm checks run	Node B (parts in hand by Oct 12, build Oct 13-18); the simulation matrix runs 10 clients
No federated-vs-local evidence	The sponsor's actual question is whether federated beats a node training alone. The fine-taxonomy sims found local-only ahead of plain FedAvg	Three arms (untrained, local-only, federated) on one frozen holdout, leave-one-site-out; add per-site fine-tuning after FedAvg as a fourth arm
Class imbalance in real data	`routine_vehicle` had 0 samples and `routine_animal` 1; balanced accuracy pinned at 0.75	Ingest street footage with `scripts/ingest_video.py --label-by-class`, then review the labels in the dashboard

10.3 Gaps in the running system

Gap	Risk	Fix
Federated and dashboard routes have no auth, even with `AUTH_REQUIRED=true`	Anyone on the network can relabel data, seed rows, or trigger training; the tunnel preflight does not notice	Add `require_auth` to the `APIRouter`s in `trial_routes.py` and `dashboard_routes.py`; probe one in `run_tunnel.sh`
Scheduler idle check is never fed	`notify_motion()` has no caller, so "only while idle" is always true	Call it from the collector or worker when motion is seen
No clip retention	`media/events/` grows until the SD card fills; a card already corrupted once	A retention sweep by age or total size (Mohammed, in progress)
Hailo unused	Every Pi number is CPU (NCNN)	Compile on the x86_64 Linux VM with `scripts/export_hailo.py --hw-arch hailo10h`; hard stop 2026-10-18, then present the CPU numbers
Dashboard does not send its token	Blocks turning auth on with the Next.js dashboard	Frontend change in `adam9798/argus-dashboard`
Tokens in memory	Every restart logs everyone out	Acceptable for the demo
Encryption at rest is a placeholder	`SecurityService.encrypt/decrypt` are TODOs	Out of scope for the showcase; state it honestly
"Package" means backpack, handbag or suitcase	Real parcels are not detected	Needs a custom-trained detector

10.4 Housekeeping

Item	Where
The GitHub default branch (`master`) is a stale mirror and `main` is behind too. The work is on `feat/threat-detection` , not merged	GitHub settings, or merge when ready
`docs/PROPOSAL.md` and `docs/SHOWCASE_PRD.md` still state 24-30 FPS; the target is now 8-12 FPS	Revise
`docs/PROPOSAL.md` still lists face requirements FR-4/FR-5/PR-4/PR-7 and objective O2 as complete	Mark "implemented, measured, disabled for performance" and keep the 0.9996 LFW result
`QUICKSTART.md` , `OVERVIEW.md` , guide 6, `FEDERATED_ARCHITECTURE.md` and `JETSON_MIGRATION.md` say rounds run only while idle / nothing is moving	True only once `notify_motion()` is wired
`tests/test_federated_infra.py::test_an_unknown_label_is_rejected` fails instead of skipping without `media/vtest.avi`	Add the same skip its sibling test has
FastAPI `on_event` is deprecated	Move to a lifespan handler when convenient
Older docs (`ARCHITECTURE.md` , `BOOTUP.md` , `WEB_APP_ARCHITECTURE.md` , `PROMETHEUS*.md`) date from January and describe the earlier design	Treat this reference as current where they disagree
Uncommitted on this laptop (2026-10-05): robust strategy wiring (`src/federated/robust_strategy.py` , `central_server/*` , `sim/fl.py` , `src/federated/server.py` , `run_central_server.ps1`), the gate loss tie-break, version archive and rollback (`model_state.py` , `trial_routes.py`), the cron retry fix (`rounds.py` , `scheduler.py`), `deploy/` , `docs/PRODUCTION_FL.md` , `ingest_video.py --label-by-class` , `scripts/measure_training_cost.py` , runbook and doc edits, and their tests	Commit and push, gated on pytest's exit code
The fine-taxonomy branch (`feat/fine-taxonomy`) exists only on the other development machine	Push it so it is not a single-machine artefact

10.5 The plan of record (from the 2026-10-04 status report)

Item	Owner	Dates
Wire `robust.py` into the Flower servers	Giovanny	**Done 2026-10-05** (uncommitted, see 10.4)
Parts for the second node ordered	Georgios	In hand by Oct 12
Alert delivery (Telegram, Discord) and dashboard label capture	Saifeddine	Oct 6 - 11
Clip retention service; login on two API routes	Mohammed	In progress
Second enclosure print and closed-case heat test	Mohammed	Before Oct 13
Federated vs. local-only comparison (three arms)	Giovanny	Oct 5 - 18
Hailo compile on the VM	Giovanny	Hard stop Oct 18
Build node B	Christian	Oct 13 - 18
Simulation matrix: 3 datasets × 3 strategies (FedAvg, FedTrimmedAvg, robust) × 3 attacks × 5 seeds, 10 clients = 135 runs	Giovanny	Oct 19 - Nov 1
Decision: two-node live demo or fallback	Team	Oct 25
Feature freeze		Nov 8
Rehearsal, poster, backup video	Team	Nov 9 - Nov 21
Showcase		Nov 22

Production operation of the federated loop (the schedule, model versions, rollback) is documented separately in `docs/PRODUCTION_FL.md`.

Datasets for the matrix, in order: CIFAR-10 with Dirichlet partitioning plus FEMNIST (comparability with published results), CASAS smart-home data (real per-home heterogeneity), then VIRAT or UCF-Crime run through ARGUS's own YOLO and `features.py` (real multi-site data in ARGUS's own feature space, filling the empty classes).

1 · What ARGUS is

A security camera that thinks on the device, and learns what is normal for the place it watches - without sending video anywhere.

The problem

A normal smart camera sends your video to a company's servers. Someone else's computer decides what matters, you pay monthly for the privilege, and it stops working when the internet does.

ARGUS keeps all of that on a small computer in your house.

What it does, in order

1. It watches. A Raspberry Pi with a camera attached, running all the time.

2. It notices movement. Comparing each frame with the one before is cheap, so this runs constantly.

3. It recognises things. A model called YOLO looks at the picture and says what is in it: a person, a vehicle, an animal, a package. It runs on the Pi itself. No internet involved.

4. It follows them. If the same person is in thirty frames, that is one event, not thirty. The system tracks each thing until it leaves.

5. It saves what matters. Each event gets a row in a small database, a still photo, and optionally a video clip with a few seconds before and after. Nothing else is kept.

6. It learns what is normal here. A small model looks at each event in context - what it was, where, how fast, how long it stayed, what time of day - and decides whether that is routine for *this* camera. A delivery van at 2pm is routine. The same van at 3am is not.

7. Cameras teach each other, without sharing video. Every so often, a camera sends what it has *learned* - a set of numbers, not footage - to be averaged with what other cameras learned. It gets the combined result back. This is called federated learning. **No image ever leaves the device.**

The pieces

Piece	Plain meaning
Node	One camera plus the Pi that runs it
Detector (YOLO)	The model that finds objects in a picture
Event	One thing that happened: a person crossed the view
The head	The small model that judges "normal here or not"
Round	One session of cameras sharing what they learned
The gate	A safety check: a shared update is only accepted if it does not make this camera worse
Hailo	An add-on chip that runs the detector far faster than the Pi's processor

---

What you see

The built-in page - open the Pi's address in a browser. Live video with boxes drawn around what it sees, the speed it is running at, and a list of recent events with photos.

The dashboard - a separate app with login, event history, and buttons to confirm or correct what the system decided. Those corrections are how it learns.

What makes it different

- **Your video never leaves the house.** Not to us, not to anyone.
- **No subscription.** You buy the hardware once.
- **It works offline.** Unplug the internet and it keeps watching.
- **It adapts to your site.** What counts as unusual at your back door is different from a driveway, and it learns that difference.

Honest limits, as of now

- **It does not recognise faces.** That was removed deliberately: it was slow, and identity is not the point. "Unusual for this site" is not "I know who that is."
- **Package means a backpack, handbag or suitcase.** The detector has no idea what a cardboard box is.
- **The fast chip is not in use yet.** The Hailo accelerator is installed and tested, but the detector still needs to be converted into its format. Until then the Pi does the work with its ordinary processor.

Next: guide 2 sets up a Pi from scratch.

2 · Setting up the Pi

From a Raspberry Pi with nothing on it, to a camera that is watching.

Every command below is labelled. **PI** means a terminal on the Raspberry Pi (a Raspberry Pi Connect shell is fine). **LAPTOP** means PowerShell on Windows. Paste whole blocks.

Before you start

You need: a Raspberry Pi 5, its camera connected by the ribbon cable, power, and a network connection. The AI HAT+ accelerator is optional at this stage.

Step 1 - system packages · PI

```
sudo apt update
sudo apt install -y git python3-venv python3-dev python3-picamera2 python3-opencv build-essential cmake
```

`python3-picamera2` is the camera library. It must come from `apt`, not from `pip` - building it yourself on a Pi rarely ends well.

Step 2 - get the code · PI

```
git clone https://github.com/Gvictome/argus.git ~/argus
cd ~/argus
git checkout feat/threat-detection
```

The branch matters. The repository's default branch is old and does not have any of this.

Step 3 - Python environment · PI

```
cd ~/argus
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

`--system-site-packages` is not optional - without it the environment cannot see the camera library you installed in step 1, and the camera will never start.

Expect this to take a while. It downloads a lot.

Step 4 - check the camera · PI

```
python -c "from picamera2 import Picamera2; print('camera library ok')"
rpicam-hello --list-cameras
```

You want the words `camera library ok` and at least one camera listed.

If the first line complains about numpy, this fixes it:

```
pip install --ignore-installed "numpy>=2.2,<3"
```

If no cameras are listed, power the Pi off and reseat the ribbon cable at both ends. Nothing further will work until this passes.

Step 5 - make detection fast · PI

```
cd ~/argus && source .venv/bin/activate
python scripts/export_cpu_models.py --format ncnn
```

This converts the detector into a format the Pi's processor handles much better. It takes about a minute and you only do it once. It is picked up automatically from then on.

Step 6 - set a password · PI

```
python scripts/set_password.py --user admin
```

It asks twice and shows nothing as you type. You need this to log into the dashboard later.

Step 7 - start it · PI

```
cd ~/argus
./scripts/run_node.sh
```

It prints the address to open, then stays running. **Leave this window alone** - closing it stops the camera.

Step 8 - confirm it is alive · PI

Open a second terminal:

```
cd ~/argus
./scripts/node_status.sh
```

One line per second. What to look for:

Reading	Means
`fps=` above 0	It is processing frames
`tracks=` rising when you step in front	It sees you
`events=` rising after you leave	It saved what it saw
`CAMERA:` appearing	Something is wrong with the camera

Press `Ctrl+C` to stop watching. That does not stop the camera.

Step 9 - look at it

Open `http://<your-pi-address>:8000/dashboard` in a browser on any machine that can reach the Pi. Find the address with:

PI

```
hostname -I
```

Use the first number, for example `10.0.0.140`, so the address is `http://10.0.0.140:8000/dashboard`.

If the Pi and your laptop are on different networks

This is common when using Raspberry Pi Connect. Your browser cannot reach the Pi directly, so:

- Everything still works from the Pi's own shells.
- To see it in a browser, either use Pi Connect's screen sharing and open the address on the Pi itself, or put both machines on the same network.

Next: guide 3 runs the demo.

3 · Running the demo

What to run, in order, and what to say while it runs.

PI = a terminal on the Raspberry Pi. **LAPTOP** = PowerShell on Windows.

Rehearse this once before the day. The first run always finds something.

Shell 1 - start the camera · PI

```
cd ~/argus
./scripts/run_node.sh
```

Leave it running for the whole demo.

Shell 2 - the live numbers · PI

```
cd ~/argus
./scripts/node_status.sh
```

Put this window where people can see it. It updates every second and it is the proof that everything else is real.

The screen - open the page

`http://<pi-address>:8000/dashboard`

Live video with boxes, the speed, what is in frame, and recent events with photos.

The walkthrough

1. Show it watching

Point at the live view. Walk into frame.

> "It is running the detection on the Pi itself. Nothing is going to a server - no internet needed."

The `tracks` number in shell 2 rises the moment you appear.

2. Show it remembering

Walk out of frame. An event appears in the list with a photo of you.

> "One event per person, not one per frame. It saved a still, and the event went into a database on the device."

3. Show what it knows

Have a backpack ready, and a phone with a photo of a car on it.

> "It knows four things: people, vehicles, animals, and packages."

Hold the backpack up. Hold the car photo up, filling the screen.

If something does not register: hold it closer so it fills a third of the view, and hold still for two seconds. A cardboard box will not work - the detector has no idea what one is.

4. Show the learning · PI, shell 3

```
cd ~/argus
./scripts/fl_trial.sh
```

While it runs:

> "Each camera trains on its own events and sends only what it learned - numbers, not footage. Those get averaged with other cameras and sent back. No video ever leaves the device."

Point at shell 2 while this happens:

> "The frame rate is not dropping. Training runs on a separate core, so the camera never stops watching to learn."

When it finishes it prints whether the update was **accepted** or **rejected**:

> "It only accepts the shared model if it does not make this camera worse on its own data. A rejection is the safety check working, not a failure."

5. Show the privacy claim

> "Everything you have seen - detection, storage, learning - happened on a £80 computer in this room. The only thing that ever leaves is a set of numbers, and you cannot reconstruct a picture from them."

What to say if asked

"How accurate is it?" On a standard benchmark image the detector is confident and correct. On the learning side, accuracy is measured against events labelled by a person, and the number is shown when a round runs. We do not quote a single headline figure because it depends on the site.

"Is that real-time?" Yes - the frame rate on screen is measured, not a target.

"Does it recognise me?" No, and deliberately. No face recognition at all. It flags things that are *unusual for this location*, which is a different claim and a safer one.

"What is the accelerator for?" It runs the detector far faster than the Pi's processor. It is installed and tested at 76 frames per second. Connecting it to ARGUS itself is the next piece of work.

Before you present

- Seeded training data is **synthetic**. Say so. The honest sentence is: "it captures and trains on real events, demonstrated alongside a synthetic set large enough to train on."
- Frame rates measured on a laptop are not Pi numbers. Quote what shell 2 shows.
- If recording is on, check disk space. Clips are a few megabytes each.

Next: guide 4 covers the dashboards.

4 · The dashboards

There are two, and they do different jobs.

The built-in page - nothing to install

`http://<pi-address>:8000/dashboard`

Served by the Pi itself, so it needs no setup, no login, and no separate program. It is the operator's view of a camera that is running right now.

What you see	What it means
Live video with boxes	What the camera sees, labelled
Detect FPS	Frames actually analysed per second, updated every second
Capture FPS	Frames arriving from the camera
ms/frame	How long one frame takes to analyse
In frame	How many things are being tracked right now
Events	How many have been saved since it started
Model	Which version of the learned model is live
A green or amber badge	Amber says "Detecting 2" when things are in view

Use this one for demos and for checking the camera is healthy.

The product dashboard - runs on your laptop

A separate app (`adam9798/argus-dashboard`) with login, event history, and the buttons that teach the system.

Start it · LAPTOP

```
cd $HOME\argus\argus
.\scripts\run_dashboard.ps1 -Node 10.0.0.140
```

Replace the address with your Pi's. The script clones it if needed, installs it the first time, points it at the Pi, and starts it. Then open `http://localhost:3000` .

Log in as **admin**, with the password you set on the Pi.

What each part does

Status strip - frame rate, accelerator, model version, cameras online. Three states: connecting, online, offline.

Event list - every saved event, newest first. Red marks something the system judged unusual for this location.

Detail panel - click an event to see everything about it, and two buttons:

- **Confirm** - the system got it right.
- **Correct** - it got it wrong. Type what it actually was, in plain English: "false alarm", "delivery driver", "suspicious person". It understands ordinary phrases.

Those two buttons are how it learns. Every click becomes a labelled example that the next training round uses. Without them, the system has only its own guesses to learn from, which teaches it nothing.

Things that commonly go wrong

"SYSTEM OFFLINE" but the Pi is fine. Two causes:

1. Your laptop cannot reach the Pi at all. Test it · LAPTOP: `powershell curl.exe -s -m 5 http://10.0.0.140:8000/api/status` No answer means different networks - use the built-in page instead.
2. The port. If port 3000 was busy, the dashboard quietly uses **3001**, and the Pi only allows the exact addresses it was told about. Check the address bar. The startup script already permits both.

The event list does not update. It loads once when the page opens. Refresh it.

Login fails. The password lives on the Pi. Set it there:

PI

```
cd ~/argus && source .venv/bin/activate
python scripts/set_password.py --user admin
```

Which to use when

Situation	Use
Demo, or checking the camera works	Built-in page
Reviewing history, labelling events	Product dashboard
Laptop cannot reach the Pi	Built-in page, on a machine that can
Showing the product to someone	Product dashboard

Next: guide 5 explains what it detects and how to test it.

5 · What it detects, and how to test it

The four things it knows

Class	What sets it off
person	a person
vehicle	bicycle, car, motorcycle, bus, train, truck
animal	bird, cat, dog, horse, sheep, cow
package	backpack, handbag, suitcase

Everything else the detector could recognise - chairs, bottles, phones, about sixty others - is thrown away before it reaches you. Those four are what a security camera cares about.

A cardboard box is not on the list. The detector was trained on a standard set of photographs that has no "parcel" in it. A real delivery box will usually detect as nothing at all. Use a backpack to demonstrate the package class. Teaching it real parcels means training a custom model, which is a separate piece of work.

How something becomes an event

1. **Movement is noticed.** Comparing consecutive frames is cheap, so this runs constantly.
2. **The detector runs.** By default only on frames where something moved.
3. **The thing is followed** from frame to frame.
4. **When it leaves, one event is saved** - not one per frame. A person crossing the view is a single event.
5. **Anything lasting under three frames is thrown away** as a flicker.

The one that catches people out

By default the detector only runs when something is **moving**. Hold a backpack perfectly still in front of the camera and nothing happens at all, even though the same picture as a file detects fine.

To classify things that hold still · PI:

```
cd ~/argus
./scripts/run_node.sh --every-frame
```

This costs speed - every frame goes through the detector - but it is the right setting when testing with props.

Testing what it sees

The direct check · PI

```
cd ~/argus && source .venv/bin/activate
python scripts/check_classes.py --camera --save /tmp/seen.jpg
```

This takes a frame from the running camera, runs the detector on it, and prints what it found. It also saves a copy with the boxes drawn on, which you can open and look at.

Output looks like:

```
ARGUS classes: {'human': 4, 'vehicle': 1}
COCO labels:   {'person(0)': 4, 'bus(5)': 1}
```

The top line is ARGUS's four classes. The bottom line is what the detector actually called them. That distinction tells you whether something was missed, or found and put in a different bucket.

Test without the camera · PI

```
python scripts/check_classes.py media/vtest.avi
python scripts/check_classes.py some-photo.jpg
```

Useful for proving the detector works when the camera is not cooperating.

Making props work

Problem	Fix
Too small in frame	Hold it closer - roughly a third of the view
Moving too fast	Hold still for two seconds
Using a phone screen	Full brightness, tilt away from lights, fill the screen
Holding it still, nothing happens	Restart with `--every-frame`
Using a cardboard box	Use a backpack instead

What gets saved

Thing	Where it goes
The event and its measurements	a small database file on the Pi
A still photo, taken when the subject was biggest	`data/snapshots/`
A video clip with a few seconds either side	`media/events/`
Long stretches of movement with nothing identified	logged separately, in chunks

The photo is taken at the subject's largest, not at the end - by the time something leaves the view it is usually half out of frame.

Clips only record if recording is on, which it is by default with `run_node.sh`. Keep an eye on disk space if you leave it running overnight.

Next: guide 6 explains the learning.

6 · How the learning works

The part that makes ARGUS more than a camera with a detector attached.

The idea in one paragraph

Several cameras, in different places, each learn from what they see. Instead of sending their video somewhere to be pooled, each one sends only **what it learned** - a list of numbers. Those get averaged and sent back, so every camera benefits from what the others saw, and **no footage ever leaves any device**. That is federated learning.

What exactly is being learned

Not "what is a person" - the detector already knows that, and it does not change.

What is learned is **what is normal at this particular camera**.

A small model looks at each event and its circumstances:

- what it was (person, vehicle, animal, package)
- how big it appeared, and where in the frame
- how fast it moved, and how long it stayed
- what time of day and day of week

and answers one question: **is this routine here, or unusual?**

A van in the driveway at 2pm is routine. The same van at 3am is not. A person at the front door at noon is routine. Someone standing at the side of the house for four minutes is not. That judgement depends entirely on the location, which is exactly why each camera has to learn its own - and why sharing what they learn is worth doing.

Where the answers come from

The system cannot mark its own homework. Labels come from a person, through **Confirm** and **Correct** in the dashboard. Every click becomes an example.

This is the honest part of the design: if the system generated its own labels from a rule, then measured itself against that same rule, the resulting accuracy figure would mean nothing.

A round, step by step

1. The camera trains on its own labelled events.
2. It sends the **updated numbers** - never images - to a coordinator.

3. The coordinator averages everyone's numbers.
4. The averaged result comes back to each camera.
5. **The gate:** each camera tests the shared model against its own held-out examples. If it does worse than the one already running, it is **rejected** and the camera keeps what it had.

Step 5 matters. An average that suits four other sites can be wrong for yours. Without the gate, one bad round quietly degrades a working camera.

The two halves

The camera trains and sends. **The central server** averages and keeps the history. They are different machines.

The central server · LAPTOP

```
cd $HOME\argus\argus
.\scripts\run_central_server.ps1
```

It prints the two lines to set on the Pi. It keeps every version of the combined model in `central_server/checkpoints`, and it **stays up between sessions** - a camera on a fortnightly schedule connects days later, so a server that exited after the first session would not be there when it mattered. Each session resumes from the last checkpoint.

Watch it at `http://localhost:8090/api/training/summary`, which lists every round with its loss and accuracy, and `http://localhost:8090/api/nodes`, which lists the cameras that have trained and what they scored.

The camera · PI

```
export FL_ENABLED=true
export FL_SERVER_URL=192.168.1.50:8080        # the laptop
export FL_CENTRAL_API=http://192.168.1.50:8090
./scripts/run_node.sh
```

With `FL_ENABLED=true` the node runs rounds **on its own schedule** - every 14 days at 2am by default, only while nothing is moving, and it catches up if the Pi was switched off across the due date. That is the cron.

Running one by hand · PI

The node must already be running in another shell.

```
cd ~/argus
./scripts/fl_trial.sh
```

That does everything: starts a coordinator, prepares training data, trains locally, joins the round, and reports the result. It runs **exactly** the same round the schedule runs, so the demo and the unattended 2am version cannot drift apart.

Training on saved video · PI

Recordings are training data. Replay a clip through the detector and label everything it finds in one go:

```
python scripts/ingest_video.py media/events/primary_human_2026.mp4 --label routine_person
python scripts/ingest_video.py trespasser.mp4 --label anomaly --every-frame
```

Each event comes out the same shape the live camera makes: the same measurements, a still, and a crop of the subject. Timestamps come from the clip's own frame rate, so a two-minute video replayed in fifteen seconds does not record everything as moving four times too fast.

Reading the result

Field	Meaning
`accepted: true`	The shared model was better or equal - it is now live
`accepted: false`	It was worse here - the camera kept its own
`current`	How the running model scored
`candidate`	How the shared model scored
`model_version`	Changes to `fl-r1`, `fl-r2` … each time one is accepted

A rejection is not a failure. It is the safety check doing its job, and worth saying out loud when demonstrating.

Watch the frame rate while it trains

Training happens in a separate process, on one processor core, so the camera keeps watching throughout. Keep `./scripts/node_status.sh` visible during a round - the frame rate barely moves. On a laptop the measured cost was 0.1%.

That is the whole argument for doing this on the device at all: a camera that stops watching in order to learn is not a security camera.

About the practice data

A camera needs a few hundred labelled examples before training is meaningful. A short demo produces a dozen. So the trial script adds **synthetic practice data** - realistic made-up events - to make the round work.

Say this when presenting. The accurate sentence is: "it captures and trains on real events, demonstrated alongside a synthetic set large enough to train on." Those synthetic rows are marked as such and never appear in the event list.

What is not solved yet

- **Averaging across very different sites can hurt.** In testing, a round combining two dissimilar locations produced a worse model, and the gate correctly rejected it. Handling that properly is real research, not a setting to change.
- **More rounds help.** Three rounds starting from an untrained partner is not enough to converge.
- **Screening needs a crowd.** The server now checks every update before averaging it and drops ones that are broken, far too large, or pulling the opposite way from everyone else. That last check needs at least three cameras to mean anything; with one or two it is skipped, and the round's report says so.

Next: guide 7 is what to do when something breaks.

7 · When things break

Every failure we actually hit, and what fixed it.

PI = terminal on the Raspberry Pi. **LAPTOP** = PowerShell on Windows.

Start here · PI

```
cd ~/argus
./scripts/node_status.sh --once
```

One line tells you most of it: frame rate, whether the camera is fine, which detector is loaded, and whether anything is in view.

The camera

"camera failed to initialize"

Work through these in order · PI:

```
cd ~/argus && source .venv/bin/activate
python -c "from picamera2 import Picamera2; print('ok')"
rpicam-hello --list-cameras
```

Result	Cause	Fix
Error mentions **numpy**	A package upgrade broke the camera library	`pip install --ignore-installed "numpy>=2.2,<3"`, restart the node
`No module named picamera2`	Library missing, or the environment cannot see it	`sudo apt install -y python3-picamera2`, and rebuild the environment with `--system-site-packages`
No cameras listed	Ribbon cable	Power off, reseat both ends
`Device or resource busy`	Something else has the camera	Close it, restart the node

Everyone looks blue

Red and blue are swapped. It should be right by default; if your camera really does hand back the other order · PI:

```
CAMERA_SWAP_RB=true ./scripts/run_node.sh
```

Detection

Only people are detected

Most likely the subject is holding still. The detector only runs on frames where something moved · PI:

```
./scripts/run_node.sh --every-frame
```

Then check what the camera is actually seeing:

```
python scripts/check_classes.py --camera --save /tmp/seen.jpg
```

Open `/tmp/seen.jpg` - it has the boxes drawn on.

Nothing at all is detected

```
python scripts/check_classes.py media/vtest.avi
```

If that finds things, the detector is fine and the problem is the scene: the subject is too small, too fast, or a cardboard box (which is not something the detector knows).

If that finds nothing either, the detector did not load. Check `backend` in the status line - `none` means it failed to start.

A box appears on screen but no event is saved

Events need three consecutive frames. A brief flicker is discarded on purpose, otherwise the list fills with noise.

Speed

Frame rate is very low

Check	Meaning
`backend` says `cpu`	No optimised detector. Run `python scripts/export_cpu_models.py --format ncnn`
You used `--every-frame`	Every frame is analysed; that is the cost
You used `--threat`	The extra stage costs around 100ms per frame on the Pi
Screen sharing is on	Sharing the Pi's screen takes processing power from the camera

---

The dashboard

"SYSTEM OFFLINE" · LAPTOP

```
curl.exe -s -m 5 http://10.0.0.140:8000/api/status
```

Result	Meaning	Fix
Returns text	Network fine; wrong port allowed	Check the address bar - if it says 3001, restart the node with the startup script, which permits it
Times out	Laptop cannot reach the Pi	Different networks. Use the Pi's built-in page instead

Login fails

The password is stored on the Pi, not the laptop · PI:

```
python scripts/set_password.py --user admin
```

Events do not update

The dashboard loads them once. Refresh the page.

Learning

The round returns an error about samples

It needs a few hundred labelled examples and you have a handful. Use `./scripts/fl_trial.sh`, which prepares practice data automatically.

The round never finishes

The coordinator is not reachable. If you used the script, it starts one itself; if you started one by hand, check it is still running and that the address matches.

The update was rejected

Working as designed. The shared model scored worse on this camera's own examples, so the camera kept the one it had.

Scripts will not run

`$'\r': command not found` · PI

The files arrived with Windows line endings:

```
cd ~/argus && git pull
chmod +x scripts/*.sh
```

`No module named 'src'` · PI

You are running a script from outside the project. Run from `~/argus`, or use the versions in `scripts/`, which handle this themselves.

`ModuleNotFoundError: uvicorn` · PI

The environment is not active in that window:

```
cd ~/argus && source .venv/bin/activate
```

Last resort

Check the window running the node. Errors are printed there, and it is usually the fastest answer in the building.

8 · For developers

Where things live, how to change them, and what is genuinely unfinished.

Two repositories

Repo	Purpose	Branch rule
`Gvictome/argus`	Backend: camera, detection, events, learning, API, built-in page	Work on `feat/threat-detection`
`adam9798/argus-dashboard`	The product dashboard (Next.js)	**Work on your branch and open a pull request. Never push to its main.**

The backend repo's default branch on GitHub is `master`, which is a stale mirror with none of this work. Always check out `feat/threat-detection`.

Layout

```
src/
  camera/       opening cameras and video files, per board
  detection/    motion, YOLO, tracking, annotation, recording
    worker.py   the background loop that owns the camera
  federated/    the learned head, samples, rounds, the gate
    collector.py    detections -> events -> training rows
    model_state.py  the validation gate and version history
    node_client.py  this node's participant in a round
  api/          routes, the dashboard contract, trial endpoints
scripts/        one-command runners and tooling
sim/            the node simulator (Flower, timing model)
docs/           documentation; docs/guides are these guides
static/         the built-in live page (one HTML file)
```

Running it

Task	Command
Node, on a Pi	`./scripts/run_node.sh`
Node against a video, anywhere	`python scripts/mock_demo.py --reset`
Tests	`python -m pytest -q`
A federated round	`./scripts/fl_trial.sh`
What the detector sees	`python scripts/check_classes.py --camera`
Build these PDFs	`python scripts/build_docs_pdf.py --guides`

The suite is 303 tests, about 100 seconds; most of that is three app startups and one video ingest. A clean run needs `pytest-asyncio` and `ultralytics` installed and `media/vtest.avi` present. See `docs/reference/09-tests.md`.

Gate any push on the test command's exit code, not on reading its output. A grep for "failed" matches a passing summary line too, which is how two failing tests once got pushed.

Settings that change behaviour

Variable	Default	Effect
`MOTION_GATE`	`true`	`false` runs the detector on every frame
`OBJECT_MODEL`	`yolov8n`	Larger models need a matching export
`OBJECT_BACKEND`	`auto`	Picks an optimised export if present
`CAMERA_SOURCE`	camera	A video file, for repeatable runs
`ARGUS_DATA_DIR`	`data`	Moves database, samples and model together
`CAMERA_SWAP_RB`	`false`	Only if colours look wrong
`THREAT_ENABLED`	`true`	Costly stage; off for demos

Design decisions worth knowing before changing things

The camera is owned by one background worker. Nothing else may read it directly - a second reader gets empty frames. Anything needing a picture goes through the worker's latest frame.

One event per track, not per frame. Otherwise a person standing still generates hundreds of near-identical rows and teaches the model that whatever loiters longest matters most.

The still is captured at the subject's largest, not when the track closes - by then they are half out of frame.

A shared model must pass the gate. Never apply a federated result directly.

Exports have a fixed input size. Asking a 640-pixel export for 416 returns nothing at all, very quickly - which reads as an enormous speed-up. The code now forces the export's own size.

What has actually been measured

On a laptop, limited to four cores, on a pedestrian video:

Change	Result
Removing face recognition	0.86 → 23.4 frames per second
Smaller working frame for motion	17.8 → 22.9 at 1080p
Smaller detector input	~60% faster, ~5% fewer detections

On the Pi, only two things are measured: the accelerator runs a large detector at 76 frames per second through its own tool, and the converted detector works. No end-to-end frame rate on the Pi has been recorded. Do not quote laptop numbers as Pi numbers.

Open work

1. **Connect the accelerator.** ARGUS does not use the Hailo yet. It needs the detector compiled into the chip's format, which requires an x86 Linux machine. Until then the Pi uses its processor.
2. **Measure on the Pi.** Every performance number so far is from a laptop.
3. **Parcels.** "Package" currently means backpack, handbag or suitcase. Real doorstep parcels need a custom-trained detector.
4. **Averaging across dissimilar sites** produced a worse model in testing. The gate catches it; making it actually work is research. The aggregator now screens every update before averaging (`src/federated/robust_strategy.py`), but with fewer than three nodes it can only catch broken or oversized updates, not disagreeing ones.
5. **Dashboard authentication.** The dashboard does not send its token to the backend. Fine on a private network, must be fixed before exposing anything to the internet.

Documentation

- `docs/pdf/ARGUS_Master_Reference.pdf` : everything below in one PDF (`python scripts/build_docs_pdf.py --master`)
- `docs/reference/` : the technical reference, covering every module, route, setting and test, and the known gaps
- `docs/guides/` - these guides, plain language, one topic each
- `docs/QUICKSTART.md` - every command, labelled by machine
- `docs/OVERVIEW.md` - the system in one pass, for a technical reader

- `docs/DEMO_RUNBOOK.md` - the full runbook with measured numbers
- `sim/README.md` - the simulator, and what it can and cannot prove

ARGUS quick start

Every block says which machine it runs on. **PI** is a Raspberry Pi Connect shell; **LAPTOP** is PowerShell on Windows.

---

1 · PI - start the node

```
cd ~/argus
git pull
./scripts/run_node.sh
```

It prints the address to open and then stays running. Leave the window alone.

Want	Command
Classify subjects that hold still	`./scripts/run_node.sh --every-frame`
Bigger detector	`./scripts/run_node.sh --model yolov8s`
Run against a recording, no camera	`./scripts/run_node.sh --video media/vtest.avi`
Add the dangerous-person stage	`./scripts/run_node.sh --threat`

First run only, to get the fast CPU path:

```
python scripts/export_cpu_models.py --format ncnn
```

---

2 · Watch it

Open `http://<pi-address>:8000/dashboard` in any browser that can reach the Pi. Live feed with boxes, detection FPS every second, what is in frame, and events with the still saved for each.

Or from a shell:

PI

```
cd ~/argus && ./scripts/node_status.sh
```

One line per second:

```
fps= 9.9 ms=   27 cap= 10.0 | tracks= 2 events=  14 motion=  3 stills=  14 | cpu-ncnn yolov8n gate=True
rate=0.98 | fl-r2
```

`tracks` rises when something is in frame; `events` rises once it leaves.

3 · PI - check what the camera sees

```
cd ~/argus && source .venv/bin/activate
python scripts/check_classes.py --camera --save /tmp/seen.jpg
```

Prints the ARGUS classes and the raw COCO labels behind them, and writes an annotated copy you can open. Point it at a file instead to test without the camera:

```
python scripts/check_classes.py media/vtest.avi
```

The four classes: person · vehicle (bicycle, car, motorcycle, bus, train, truck) · animal (bird, cat, dog, horse, sheep, cow) · package (backpack, handbag, suitcase). COCO has no parcel class, so a cardboard box detects as nothing - use a backpack.

---

4 · PI - a federated round

Node running in shell 1, then:

```
cd ~/argus && ./scripts/fl_trial.sh
```

It starts the aggregator, seeds samples, trains locally, joins the round, and reports the gate's decision. Watch `detect_fps` while it runs - training is a separate process on one core, so detection keeps going.

`model_version` moves to `fl-r1`, `fl-r2` … only when the gate **accepts** the averaged model. A rejection means it scored worse on this node's own held-out samples, and the node kept the model it had. That is the gate working.

> The seeded samples are synthetic and never appear in the event list. Say so when presenting: the pipeline captures and trains on real events, demonstrated alongside a corpus large enough to train on.

4b · The central server, and the schedule

The laptop is where the cameras send what they learned. Weights only - no video, no images.

LAPTOP - start it

```
cd $HOME\argus\argus
.\scripts\run_central_server.ps1
```

It prints the two lines to set on the Pi, keeps every version of the combined model in `central_server/checkpoints`, and stays up between sessions so a scheduled camera finds it days later.

Watch	URL
Rounds, loss, accuracy	`http://localhost:8090/api/training/summary`
Cameras and their scores	`http://localhost:8090/api/nodes`
Prometheus	`http://localhost:8090/metrics`

PI - join on a schedule

```
export FL_ENABLED=true
export FL_SERVER_URL=192.168.1.50:8080
export FL_CENTRAL_API=http://192.168.1.50:8090
./scripts/run_node.sh
```

Rounds then run by themselves: every 14 days at 2am, only while nothing is moving, catching up if the Pi was off across the due date. Check the schedule any time:

```
curl -s localhost:8000/api/federated/status | python -m json.tool
```

PI - turn saved video into training data

```
python scripts/ingest_video.py media/events/some_clip.mp4 --label routine_person
```

Labels: `routine_person`, `routine_vehicle`, `routine_animal`, `routine_package`, `anomaly`. Stop the node first - both write to the same store.

---

5 · LAPTOP - the product dashboard (optional)

```
cd $HOME\argus\argus
.\scripts\run_dashboard.ps1 -Node 10.0.0.140
```

Then open `http://localhost:3000` (or `:3001`) and log in as `admin`.

Set the password on the **PI**:

```
python scripts/set_password.py --user admin
```

If it says SYSTEM OFFLINE, your laptop cannot reach the Pi, or the port Next.js chose is missing from `CORS_ORIGINS`. The node's own page at `http://<pi>:8000/dashboard` has no such dependency.

If it breaks

Symptom	Fix
Only `person` detected	Subject is holding still → `--every-frame`; or it is too small in frame
Nothing detected at all	`python scripts/check_classes.py --camera` - separates model from scene
`camera_error` in status	`python -c "from picamera2 import Picamera2"`; if it mentions numpy → `pip install --ignore-installed "numpy>=2.2,<3"`
`ImportError: ... Bad message`	A system library is **corrupt**, not missing. See "When the card goes bad" below
People look blue	`CAMERA_SWAP_RB=true ./scripts/run_node.sh`
Round returns 409	Fewer than 200 labelled samples - let `fl_trial.sh` seed them
Dashboard offline, curl works	Wrong port in `CORS_ORIGINS` (Next.js often uses 3001)
`backend: cpu` on the Pi	`python scripts/export_cpu_models.py --format ncnn`

When the card goes bad

`Bad message` in an `ImportError` is **EBADMSG** - ext4 refusing to hand over a file whose checksum no longer matches. The file is corrupt, not missing, and reinstalling with `apt` may not even run: `apt` is C++, so a damaged `libstdc++` takes it down too. An unclean shutdown is the usual cause.

`dpkg` records an md5 for every file it installed, so the whole system can be verified offline. **PI**

```
cd / && cat /var/lib/dpkg/info/*.md5sums | md5sum -c 2>/dev/null | grep -v ': OK$' | sed 's/: FAILED.*//'
| sed 's|^|/|' > /tmp/bad.txt
wc -l < /tmp/bad.txt
```

Map those files back to their packages and reinstall the lot. The `grep -v '^diversion'` matters - `dpkg -S` emits diversion lines that are not package names:

```
xargs -a /tmp/bad.txt dpkg -S 2>/dev/null | grep -v '^diversion' | cut -d: -f1 | tr ',' '\n' | tr -d ' '
| grep -v '^$' | sort -u > /tmp/badpkgs.txt
sudo apt install --reinstall -y $(tr '\n' ' ' < /tmp/badpkgs.txt)
```

If `apt` itself will not start, repair `libstdc++` first with `dpkg`, which is plain C. Take the version from `dpkg -l libstdc++6`:

```
cd /tmp
U=http://deb.debian.org/debian/pool/main/g/gcc-14
curl -fLO $U/libstdc++6_14.2.0-19_arm64.deb
sudo dpkg -i /tmp/libstdc++6_14.2.0-19_arm64.deb && sudo ldconfig
```

Re-run the scan afterwards. Zero failures means every tracked file matches Debian's published hash. Reboot if the kernel, `systemd`, `kmod` or the Hailo driver were among them, then confirm with `hailortcli fw-control identify`.

A file that fails again after being rewritten means the card is bad - reimage, and do not reuse it. Check `dmesg | grep -iE 'mmc0|I/O error'` : a silent failure with no I/O errors logged is typical of a counterfeit or worn card.

ARGUS node simulator

A single ARGUS node running its detection loop at a target frame rate while a **real Flower federated round** runs against it, so you can watch what the round does to the frame rate.

```
python sim/run_node.py                                  # one node, 12 FPS
python sim/run_node.py --clients 3 --rounds 6           # real Flower rounds, robust aggregation
python sim/run_node.py --profile pi5-threat-on-cpu      # the risk case
python sim/run_node.py --pipelined                      # overlap the Hailo
python sim/run_node.py --cores 4                        # match Pi core count
```

Metrics land on `http://127.0.0.1:9108/metrics` (`--port 0` to disable).

The question it answers

`ARGUS_Materials_List_B_Sponsor_Spec.md` rejected every 8 GB board against "REQ 3", on the claim that under 16 GB "*a live security node stops detecting while it trains. That is a functional failure, not a slowdown.*"

That claim was written about **Jetson unified memory**, where CPU and GPU share one pool and a Flower client holding GPU memory genuinely starves inference. The architecture actually being built is different: detection is offloaded to the Hailo-10H, which has its own 8 GB, and the trainer is a ~200K parameter head on the CPU. This simulator measures whether REQ 3 transfers. It does not.

Why it is not just a spreadsheet

The thing that makes the numbers mean anything:

Stage type	What the sim does	Why
`cpu`	calibrated numpy matmul loop	Burns a real core. Contends with the trainer through the OS scheduler, exactly as the real pipeline would.
`accel`	`time.sleep()`	The Hailo works over PCIe; the CPU is free meanwhile. Sleeping is the honest model.
`measured`	real forward pass, timed	The federated head is a real model, not a budget.

If CPU stages were also sleeps, a concurrent training round would show zero impact - which is the entire question. The trainer runs as a **separate OS process** talking gRPC, not a thread, so contention is real.

> **One BLAS thread per process is mandatory** and is forced at the top of `run_node.py` and `fl.py`. numpy's BLAS is multithreaded by default, so a single "CPU stage" would otherwise saturate every core; with the trainer doing the same the two oversubscribe the machine and both collapse. The first run of this sim showed 393 ms for a 12 ms sleep before this was fixed.

Profiles

Timings come from Rev 2.2 Table 11 and `docs/HAILO_PIPELINE.md`, **with ArcFace removed** per the 2026-09-14 scope change. Each stage carries its source.

Profile	CPU	Accel	Serial ceiling	Pipelined
`pi5-hailo` (target build)	27 ms	40 ms	**14.9 FPS**	25.0 FPS
`pi5-threat-on-cpu` (risk)	63 ms	28 ms	**11.0 FPS**	15.9 FPS
`pi5-cpu-only` (fallback)	405 ms	0 ms	2.5 FPS	2.5 FPS
`x86-dev` (measured)	140 ms	0 ms	7.1 FPS	7.1 FPS

`pi5-threat-on-cpu` is the case `HAILO_PIPELINE.md` §3 warns about: an uncompiled threat model is "*a second CPU YOLO pass at ~36 ms/frame*". Table 11 has no threat line at all, so this is the gap between the proposal and the code.

The federated head

Replaces the CIFAR-10 stand-in. It does not classify pixels - YOLO already emits person/vehicle/animal/package, and reproducing that adds nothing. It classifies a **detection in context**: what was found, how big, where, how long it lingered, what time it was. A delivery van at 14:00 is routine; the same van at 03:00 is not. That judgement is site-specific, which is what makes it worth federating.

- 19 features → 384 → 384 → 128 → 5 classes, **205,445 params** (CON-3 bound ~200K)
- Labels are the five from `flower_training_report_2026-04-24.md:18`: routine person / vehicle / animal / package, plus anomaly
- Pure numpy, so `NumPyClient` is a direct fit and the RSS figure stays honest

Genuinely non-IID

`flower_training_report_2026-04-24.md:36` partitioned CIFAR-10 with `IidPartitioner` - "*each node gets a random, balanced sample*". That is the easiest possible split; it validates plumbing, not learning.

Here each of the three sites has both a different **label** distribution and a different **conditional** distribution - the anomaly rule itself differs:

Site	Mix	Rule
`driveway`	vehicles + people all day	night presence is the signal
`backdoor`	packages + staff	a vehicle here is *always* wrong
`yard`	mostly animals	tolerant of loitering, not of people at night

Watch the per-site balanced-accuracy spread in the report. That spread is the non-IID effect, and it is the thing a 5-node deployment has to survive.

Reading the output

Three phases, same pipeline throughout:

```
phase         frames     fps   p50 ms   p95 ms   p99 ms
baseline          96   14.44     69.2     70.1     76.7
fl               510   14.19     68.7     76.4    112.9  (-1.7%)
recovery          72   14.44     68.9     72.1     77.6  (+0.0%)
```

`fl` is the window where the Flower round was running. The delta against baseline is the answer to REQ 3.

The stage table compares measured against budget, so you can see which budget lines the simulator actually reproduces and which drift.

What this does not model

Stated plainly because the numbers will get quoted:

- **CPU stage times are budgets from Rev 2.2 Table 11, not Pi measurements.** No Pi 5 + Hailo-10H benchmark exists in this repo yet. `DETECTION_ACCURACY.md` §6 is explicit that its x86 figures are "*a ceiling, not a prediction*".
- **The head is numpy.** Keeping TensorFlow in the federated loop adds roughly 1 GB of framework RSS this does not model.
- **These are this machine's cores, not the Pi's.** `--cores 4` matches the count, not the per-core performance.
- **Memory pressure is not simulated.** The sim shows CPU contention, which is what actually binds here; it is not evidence about 8 GB vs 16 GB on its own.

To replace the budgets with real numbers, run `scripts/benchmark_classes.py` on the Pi and edit `sim/profiles.py`.

Files

File	Role
`profiles.py`	Hardware timing profiles, every line sourced
`workload.py`	Non-IID per-site event generator
`head.py`	The ~200K param federated head (numpy MLP)
`pipeline.py`	Frame loop; real CPU burn vs accelerator sleep
`fl.py`	Flower server + client, run as separate processes
`run_node.py`	Orchestrator, Prometheus surface, report
`_run/fl_events.jsonl`	Raw per-round events from the last run